

SIP Client API Specification

Version 1.0; July 5, 2004

SIP

NOKIA

Copyright © 2004 Nokia Corporation. All rights reserved.

Nokia and Nokia Connecting People are registered trademarks of Nokia Corporation. Java and all Java-based marks are trademarks or registered trademarks of Sun Microsystems, Inc. Other product and company names mentioned herein may be trademarks or trade names of their respective owners.

Disclaimer

The information in this document is provided “as is,” with no warranties whatsoever, including any warranty of merchantability, fitness for any particular purpose, or any warranty otherwise arising out of any proposal, specification, or sample. Furthermore, information provided in this document is preliminary, and may be changed substantially prior to final release. This document is provided for informational purposes only.

Nokia Corporation disclaims all liability, including liability for infringement of any proprietary rights, relating to implementation of information presented in this document. Nokia Corporation does not warrant or represent that such use will not infringe such rights.

Nokia Corporation retains the right to make changes to this specification at any time, without notice.

License

A license is hereby granted to download and print a copy of this specification for personal use only. No other license to any other intellectual property rights is granted herein.

Contents

1	Introduction.....	6
1.1	List of Required Interfaces	6
1.2	Product Platform and/or Symbian OS Version Constraints.....	6
2	Technical Specification.....	7
2.1	Type of Interface.....	7
2.2	Interface Class Structure	7
2.3	Usage.....	11
2.3.1	Protocol	11
2.3.2	Error handling	18
2.3.3	Memory overhead.....	19
2.3.4	Extensions to the API.....	19
2.4	Example	20
2.5	Detailed Description	22
2.5.1	CSIP	22
2.5.2	CSIPClientTransaction	23
2.5.3	CSIPConnection	25
2.5.4	CSIPDialog.....	29
2.5.5	CSIPDialogAssocBase.....	32
2.5.6	CSIPInviteDialogAssoc.....	33
2.5.7	CSIPMessageElements.....	40
2.5.8	CSIPRefresh.....	43
2.5.9	CSIPRegistration	46
2.5.10	CSIPRequestElements	52
2.5.11	CSIPResponseElements.....	56
2.5.12	CSIPServerTransaction.....	60
2.5.13	CSIPSubscribeDialogAssoc.....	61
2.5.14	CSIPTransactionBase	65
2.5.15	MSIPConnectionObserver	67
2.5.16	MSIPObserver	72
2.5.17	MSIPRegistrationContext.....	73
2.5.18	SIPHttpDigest.....	74
2.6	Code Architecture	75
3	Index	77
3.1	List of Constants and Enums.....	77
3.1.1	class CSIP	77
3.1.2	class CSIPConnection	78

3.1.3	class CSIPDialog	78
3.1.4	class CSIPInviteDialogAssoc	78
3.1.5	class CSIPRefresh	78
3.1.6	class CSIPSubscribeDialogAssoc.....	78
3.1.7	class CSIPTransactionBase.....	78
3.2	Alphabetical List of Classes and Functions.....	79
3.2.1	class CSIP	79
3.2.2	class CSIPClientTransaction.....	79
3.2.3	class CSIPConnection	80
3.2.4	class CSIPDialog	80
3.2.5	class CSIPInviteDialogAssoc	80
3.2.6	class CSIPMessageElements	81
3.2.7	class CSIPRefresh	81
3.2.8	class CSIPRegistration.....	81
3.2.9	class CSIPRequestElements	82
3.2.10	class CSIPResponseElements	82
3.2.11	class CSIPServerTransaction.....	82
3.2.12	class CSIPSubscribeDialogAssoc.....	82
3.2.13	class CSIPTransactionBase.....	83
3.2.14	class MSIPConnectionObserver	83
3.2.15	class MSIPObserver	83
3.2.16	class MSIPRegistrationContext.....	83
3.2.17	class SIPHttpDigest	84
4	Terms and Abbreviations	85
5	References	86

Change History

July 5, 2004	Version 1.0	Initial document release

1 Introduction

Session Initiation Protocol (SIP) is a de facto application-layer-signaling protocol for creating, modifying, and terminating session(s) with participant(s). SIP extensions are used, for example, for messaging, subscribing to events, receiving event notifications, and publishing presence information. SIP has been adopted by the 3rd Generation Partnership Project (3GPP) and used in IMS Release 5. SIP can run on top of several different transport protocols.

The SIP Client API provides access to the basic services of a SIP stack. The most essential services are sending and receiving SIP messages, creating registrations, and forming and tearing dialogs initiated by INVITE and SUBSCRIBE.

The SIP Client API is appropriate for applications that intend to use SIP for managing multimedia sessions, subscribing to events, and sending stand-alone SIP messages.

1.1 List of Required Interfaces

The SIP Client API requires the SIP Codec API. For further details, see the *SIP Codec API Specification* [1].

1.2 Product Platform and/or Symbian OS Version Constraints

There are no constraints. This API is valid for all platforms running Symbian OS v7.0s.

2 Technical Specification

2.1 Type of Interface

The logical type of the SIP Client API is a Library API.

The technical type of the API is a Server Client API, which allows multiple clients to use the SIP stack simultaneously. The function calls of the API are synchronous. An application must implement a set of callback functions, which the SIP Client API uses to pass events to the application.

2.2 Interface Class Structure

An application using the SIP Client API will create one instance of CSIP. The application is responsible for implementing the callback functions of MSIPObserver, which are used by CSIP. With CSIP it is possible to query the supported security mechanisms and find out if there exists a CSIPConnection for a given IAP.

Once CSIP exists, it is possible to create an instance of CSIPConnection. The application must implement the callback functions of MSIPConnectionObserver. With CSIPConnection, it is possible to send stand-alone SIP requests and ask for the state of the network connection. The responses are received through MSIPConnectionObserver. The application can have multiple CSIPConnection objects, but each tone must have a different IAP.

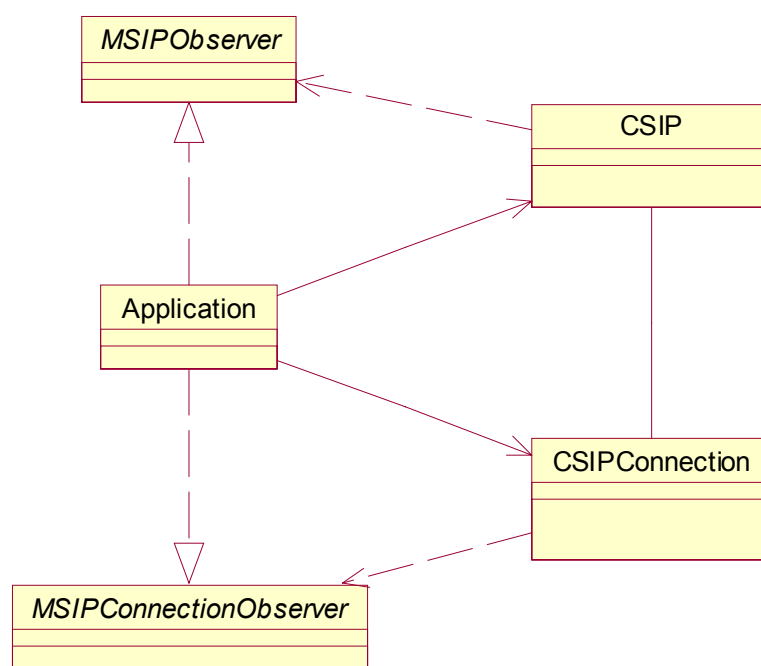


Figure 1: CSIP, CSIPConnection, and their observer classes

The CSIPRegistration class provides services for registering, updating registration, and deregistering. It is also possible to instruct the SIP stack to automatically refresh the registration, and query and set the outbound proxy used by the registration. Creating the CSIPRegistration instance requires an existing CSIPConnection.

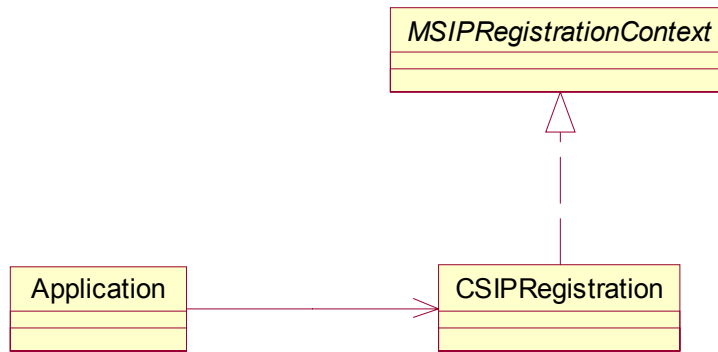


Figure 2: Registration-related classes

The `CSIPInviteDialogAssoc` class allows the application to initiate a dialog and to send INVITE, PRACK, ACK, UPDATE, and BYE requests related to the dialog. When `CSIPInviteDialogAssoc` is instantiated, it can be bound to an existing `CSIPDialog` instance, or it can be used to create a new `CSIPDialog`. `CSIPDialog` is instantiated by the SIP Client API; it cannot be created by the application.

`CSIPSubscribeDialogAssoc` is very similar to `CSIPInviteDialogAssoc`. It can also be bound to an existing `CSIPDialog`, or be used to create a new `CSIPDialog`. With `CSIPSubscribeDialogAssoc`, the application can subscribe to presence information by sending SUBSCRIBE and (un)SUBSCRIBE requests. It is also possible to order the SIP stack to automatically refresh the SUBSCRIBE request.

`CSIPDialogAssocBase` is a base class for both `CSIPInviteDialogAssoc` and `CSIPSubscribeDialogAssoc`, and provides a way to locate the associated `CSIPDialog` instance and send a nontarget refresh request within a dialog.

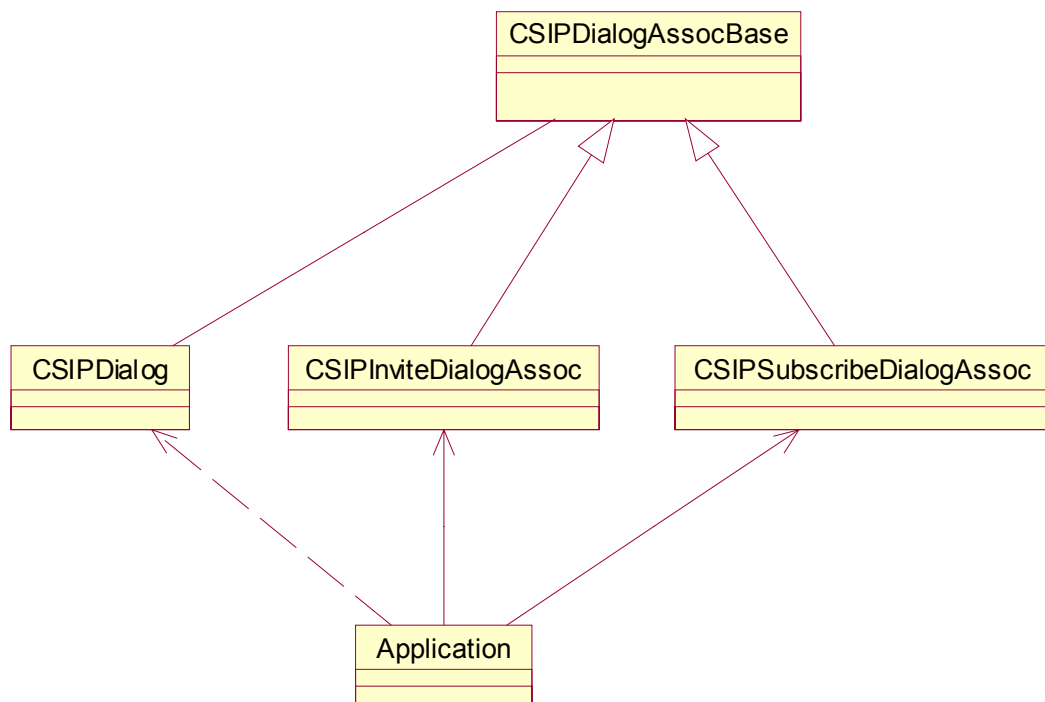


Figure 3: Dialog-related classes

When the application invokes a function, which causes a SIP request to be sent to the network, the SIP Client API implementation instantiates a new `CSIPClientTransaction` object and returns it to the application. `CSIPClientTransaction` represents the SIP transaction consisting of the SIP request and the SIP response. Once a response has been received, the application can obtain `CSIPResponseElements` from `CSIPClientTransaction`. Certain client transactions can also be canceled or refreshed.

When a SIP request is received from the network, it is passed to the application as a `CSIPServerTransaction`. The application can obtain `CSIPRequestElements`, describing what kind of SIP request was received, from `CSIPServerTransaction`. Once the application has decided what kind of response(s) it is going to send back, it uses `CSIPServerTransaction`.

`CSIPTransactionBase` provides functions for obtaining the current state of the transaction and transaction type. Some operations are only allowed when the transaction is in a certain state. `CSIPClientTransaction` and `CSIPServerTransaction` are instantiated by the SIP Client API; they cannot be created by the application.

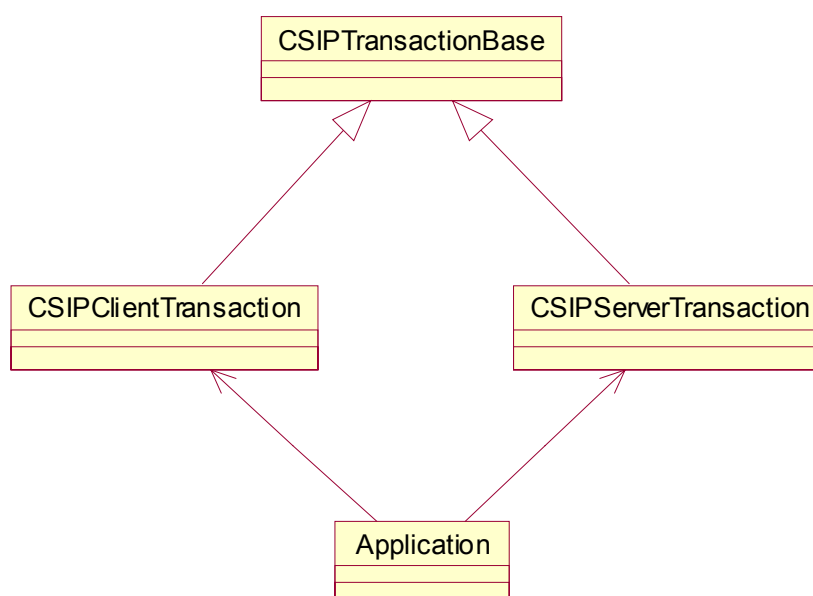


Figure 4: Transaction-related classes

The `CSIPMessageElements` class holds the parts of SIP messages that are common to both SIP requests and responses. It provides functions for getting and setting those SIP headers that are visible to the application. The contents of the SIP message can also be obtained and set. The application typically needs to instantiate a `CSIPMessageElements` object when it is going to send a SIP request.

`CSIPRequestElements` contains `CSIPMessageElements` and the mandatory parts of any SIP request. It provides setters and getters for manipulating To-header, From-header, Remote-URI, request method, and `CSIPMessageElements`. The application typically needs to instantiate a `CSIPRequestElements` object when it is going to send a SIP extension request.

`CSIPResponseElements` contains `CSIPMessageElements` and the mandatory parts of any SIP response. It provides setters and getters for the status code and reason phrase, and getters for obtaining the From, To, and CSeq-headers and `CSIPMessageElements`.

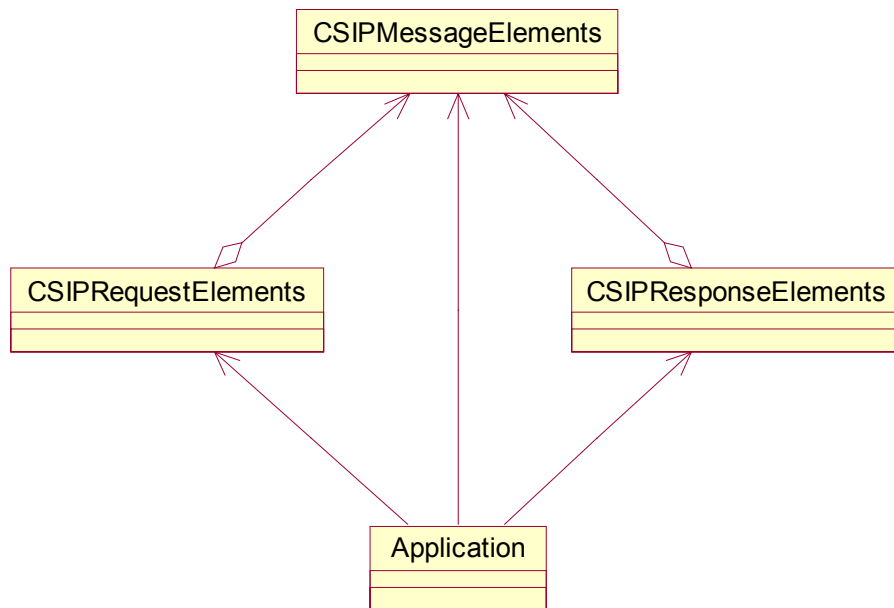


Figure 5: SIP message-related classes

The CSIPRefresh class is used when a SIP request should be refreshed automatically by the SIP stack. REGISTER, SUBSCRIBE, and a request outside of a dialog can be refreshed. Once a refreshed request has been sent, the application can query the state of the refresh from CSIPRefresh. For requests outside of a dialog, CSIPRefresh provides functions for updating and terminating the refresh. Updating and terminating a refreshed REGISTER or SUBSCRIBE is done via CSIPRegistration and CSIPSubscribeDialogAssoc classes, instead of using CSIPRefresh.

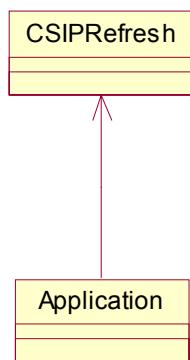


Figure 6: CSIPRefresh class

The SIPHttpDigest class contains only static functions. It provides ways to manage HTTP digest security settings. To be able to use SIPHttpDigest, an instance of CSIP is required.

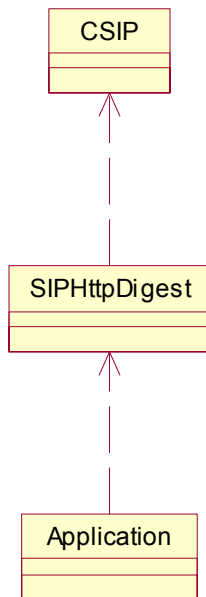


Figure 7: SIPHttpDigest class

2.3 Usage

2.3.1 Protocol

2.3.1.1 Initiating SIP Client

An application that is using the SIP Client API first creates an instance of `CSIP`. The application has to implement the callback functions of `MSIPObserver`. When `CSIP` has been created, the client is connected to a shared server. If the server is not currently running, it will start now. This may take a while, because all the server-side resources are being created and initialized.

The application creates an instance of `CSIPConnection` to be able to use an IAP. The application has to implement `MSIPConnectionObserver` callbacks. As the `CSIPConnection` is created, a subsession is formed between the client and the server.

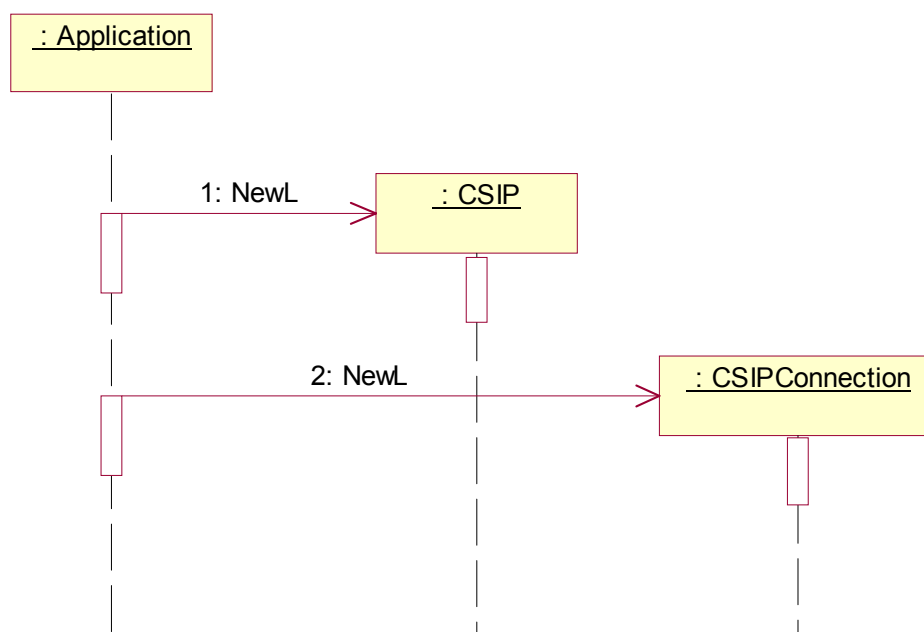


Figure 8: Initiating SIP Client

2.3.1.2 Closing SIP Client

The application can delete the SIP Client API objects in any order. An exception to this is that `CSIP` must be deleted before the associated `MSIPObserver`, and `CSIPConnection` must be deleted before the associated `MSIPConnectionObserver`.

When the `CSIP` instance is deleted, the connection to the server is closed. The server may be stopped at that point. Deleting a `CSIPConnection` will cause a subsession between the client and the server to be closed.

If the application deletes an object to which there are dependencies from other objects, this might render the other objects unusable. Objects that are in an unusable state will leave with the error code `CSIP::EResourceNotAvailable`, indicating that they cannot perform the requested operation because they no longer have access to the deleted object.

An example of the above scenario is when an application has a `CSIPConnection` and `CSIPRegistration` objects, and it deletes `CSIPConnection`. Using `CSIPRegistration` after that will, in most cases, fail, because without `CSIPConnection` there is no way for `CSIPRegistration` to communicate with the server side of the SIP stack.

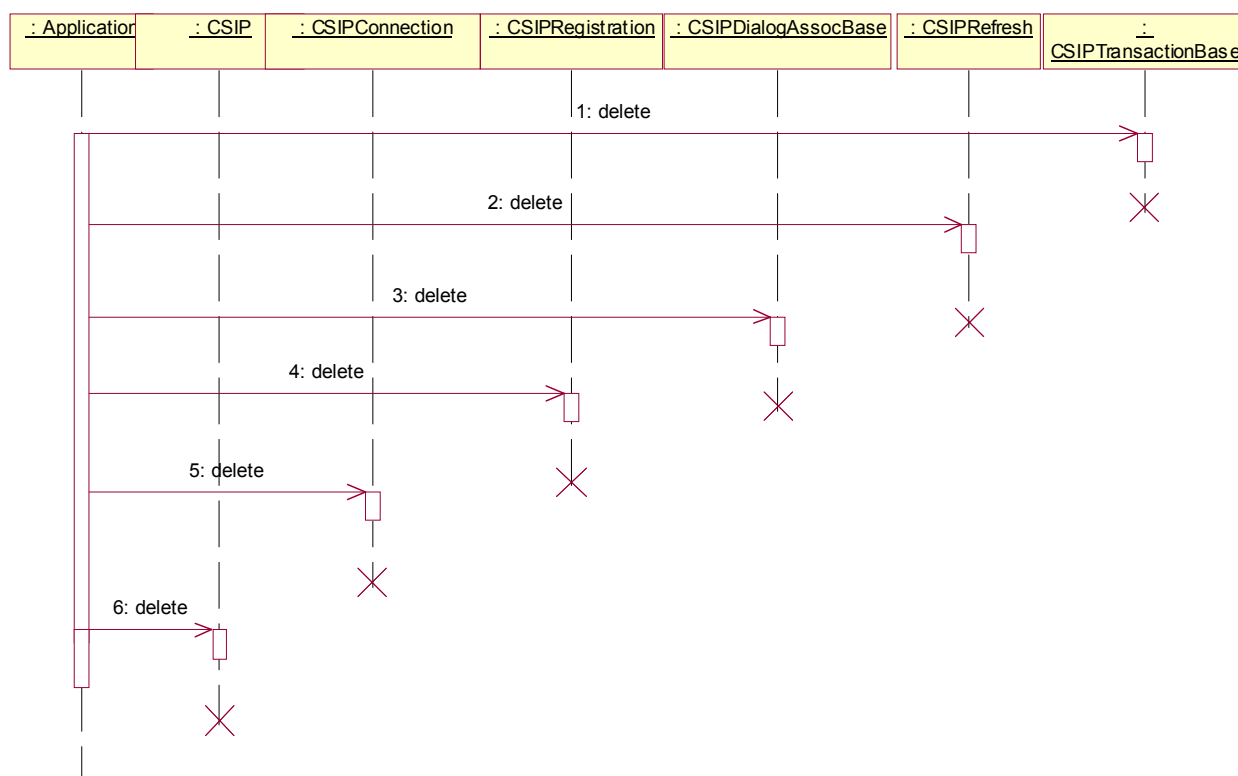


Figure 9: Recommended order for deleting SIP Client API objects

2.3.1.3 Sending a SIP request

In order to send a SIP request, the application creates an instance `CSIPRequestElements`, and sets the desired parts of the request. Once `CSIPRequestElements` has been filled, the application invokes the `CSIPConnection::SendRequestL(CSIPRequestElements*)` function. `CSIPConnection` takes ownership of `CSIPRequestElements` and communicates the request to the server side of the SIP stack. `CSIPConnection` instantiates a `CSIPClientTransaction` and returns it to the application. `SendRequestL` initiates the server-side processing of sending the MESSAGE, but time-consuming tasks like address resolving and socket operations are done only after the `SendRequestL` call has returned. If an error happens after `SendRequestL` has returned, for example, in the address resolving, the error will be communicated to the application by `MSIPConnectionObserver::ErrorOccured`. It is important to understand that when `SendRequestL` returns, the MESSAGE has not yet been sent to the network but is being processed on the server side of the SIP stack.

Once `SendRequestL` has returned and the sending of a MESSAGE is continuing on the shared server, there is no way for the application to cancel the operation. This applies to other kinds of operations, too. Note that the `CSIPClientTransaction::CancelL` function is used to send a SIP CANCEL request after the application has first sent an INVITE request.

When a SIP response message is received from the server side, `CSIPConnection` forms a `CSIPResponseElements` object to represent the SIP response, attaches it to the `CSIPClientTransaction`, and notifies the application with `MSIPConnectionObserver::IncomingResponse`. The application can determine what kind of a response was received by accessing `CSIPResponseElements` from `CSIPClientTransaction`. Because the response was a provisional response, the application does not delete `CSIPClientTransaction` yet. Later, a 200 response is received and the application deletes `CSIPClientTransaction`, which has now completed.

Figure 10 illustrates the steps of sending a MESSAGE request and receiving two responses.

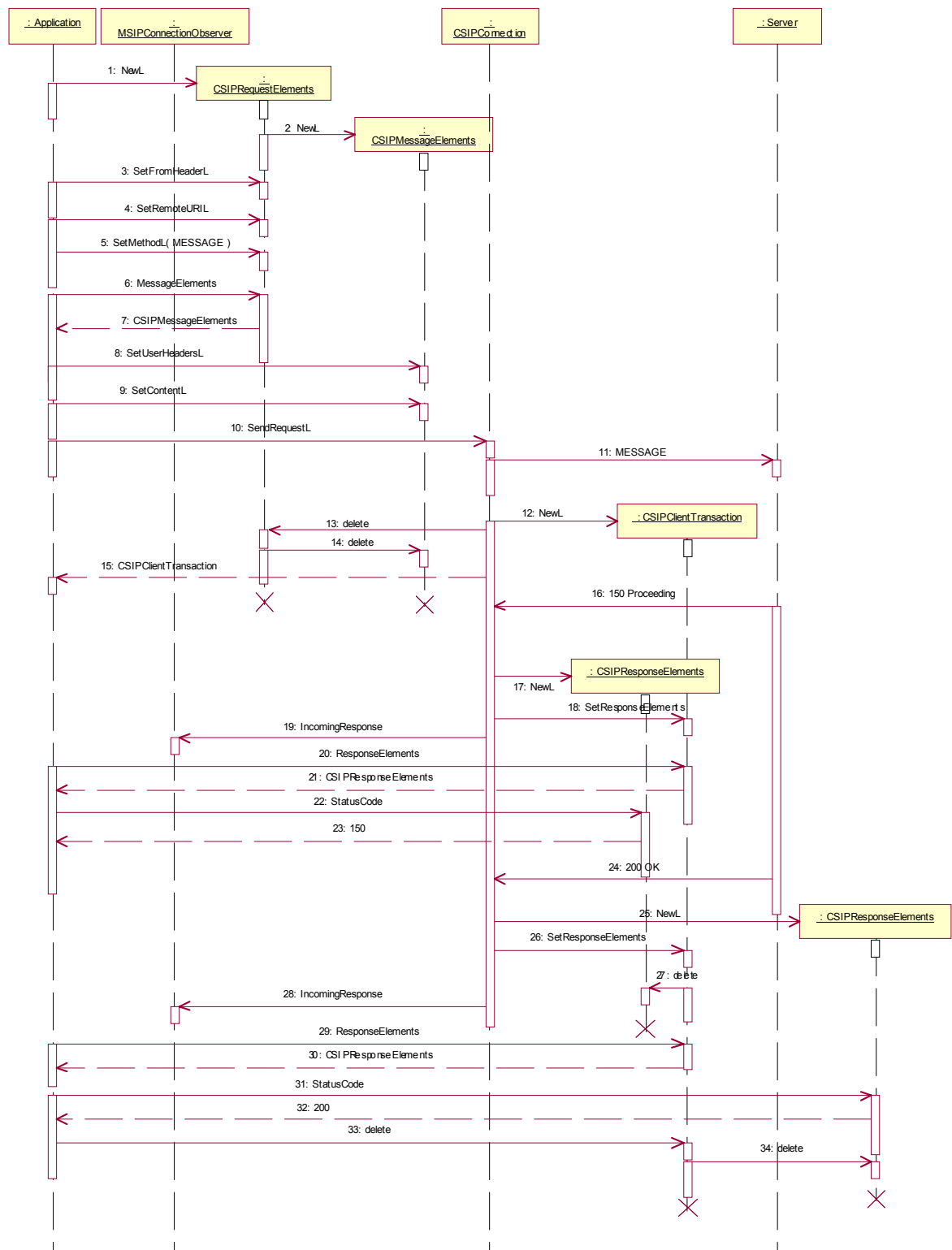


Figure 10: Sending a SIP request

2.3.1.4 Receiving a SIP request

In this example, a stand-alone SIP request is receiving through an existing CSIPConnection. CSIPConnection creates CSIPRequestElements and CSIPServerTransaction to represent the incoming SIP request, before notifying the application with an IncomingRequest event to MSIPConnectionObserver. The application obtains information about the request by getting the CSIPRequestElements of the CSIPServerTransaction. When a response is sent, CSIPResponseElements needs to be instantiated and passed to CSIPServerTransaction with the SendResponseL function. This causes the response to be handed to the server side for further processing. When SendResponseL has returned, the application has no more need for the CSIPServerTransaction and deletes it.

Figure 11 displays the steps of receiving a MESSAGE and sending a response to it.

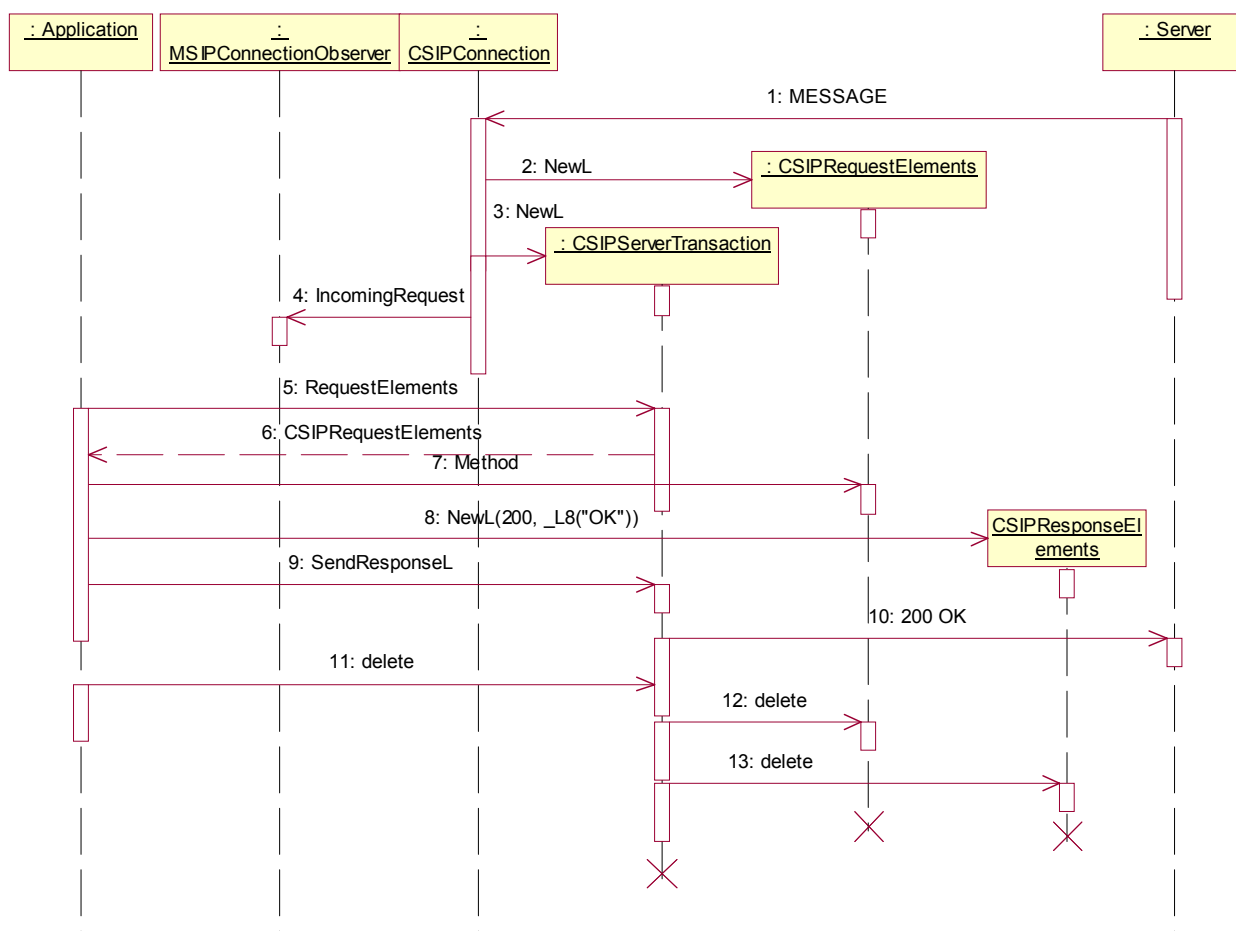


Figure 11: Receiving a MESSAGE and sending a response

2.3.1.5 Registering

To initiate a registration, the application first creates a CSIPRefresh object, and then creates a CSIPRegistration, passing CSIPRefresh to it. It is not mandatory to create CSIPRefresh. It is only needed if the registration is to be refreshed by the SIP stack. Once CSIPRegistration exists, the application can initiate registration by calling RegisterL. CSIPRegistration forms a REGISTER request and communicates it to the server side of the SIP stack, instantiates a CSIPClientTransaction, and returns it to the application.

When the 200 response is received from the server side, CSIPConnection creates CSIPResponseElements for holding the response, and routes the response to CSIPRegistration. CSIPRegistration attaches the response to CSIPClientTransaction. The application is notified about the response with a MSIPConnectionObserver::IncomingResponse. The application no longer needs CSIPClientTransaction and deletes it. It uses IsContextActive to find out if the registration has succeeded.

Figure 12 shows the steps of registering with a refresh and receiving a 200 OK response.

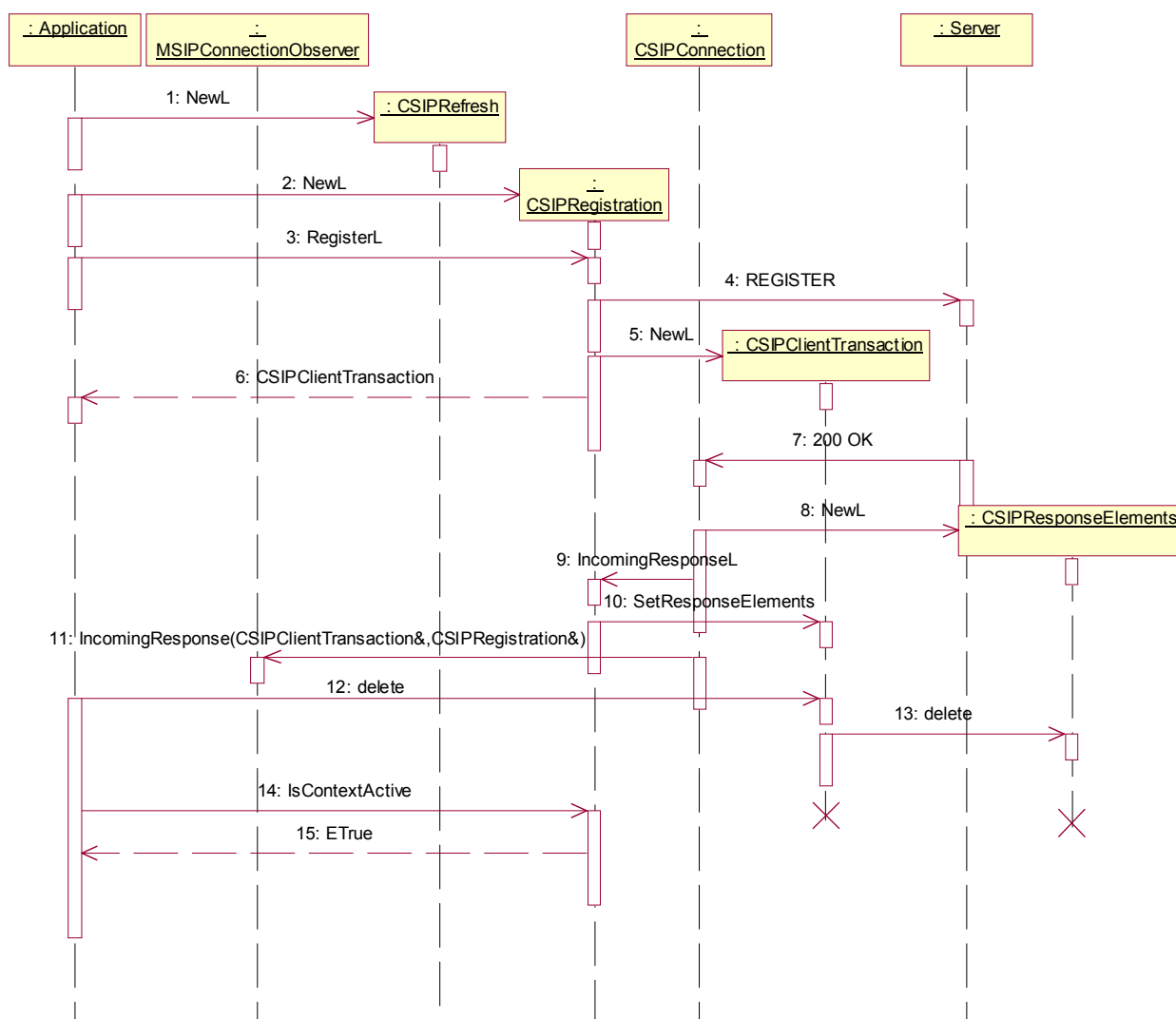


Figure 12: Registering with refresh

2.3.1.6 Creating a dialog with INVITE

In order to create a dialog, the application creates CSIPInviteDialogAssoc and CSIPMessageElements objects. The application fills the relevant SIP headers into CSIPMessageElements, before passing it to CSIPInviteDialogAssoc by using the SendInviteL function. CSIPInviteDialogAssoc forms an INVITE request and communicates it to the server side of the SIP stack, instantiates a CSIPClientTransaction, and returns it to the application.

As the 180 response is received from the server side, CSIPConnection creates CSIPResponseElements for holding the response, and routes the response to CSIPDialog. CSIPDialog attaches the response to CSIPClientTransaction. The application is informed about the reception of the 180 response with MSIPConnectionObserver::IncomingResponse.

The processing that occurs when the 200 response is received is similar to that which occurs with the 180 response. At this point, the application is expected to respond by sending an ACK request. This is done by invoking the CSIPInviteDialogAssoc::SendAckL function, which forms an ACK request and sends it to the server side of the SIP stack.

Since there is a possibility that the INVITE request might fork at a proxy, the SIP stack's server side waits a while for several possible responses. When the forking is no longer expected to happen, the SIP stack's server side sends an InviteCompleted event. The SIP Client API forwards the InviteCompleted event to the application, indicating the application can now delete CSIPClientTransaction.

Figure 13 shows the steps of sending INVITE, receiving a response, and sending ACK.

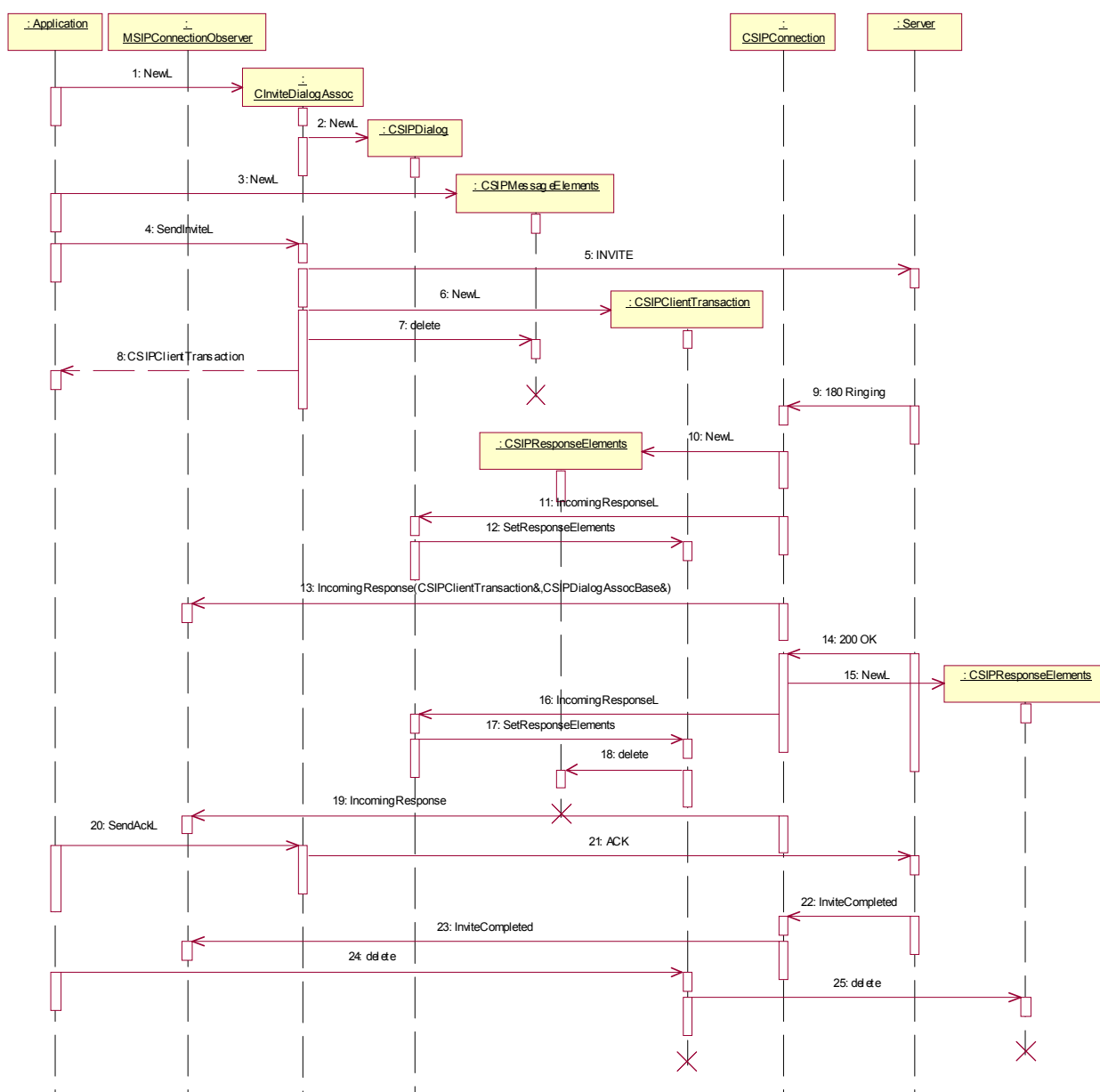


Figure 13: Creating a dialog by sending an INVITE

2.3.2 Error handling

Errors are handled to the application with leaving functions and/or with callback functions.

If the error occurs during the synchronous part of an operation initiated by the application, functions leave with the error code.

If the error occurs later during the asynchronous processing, the error is passed to the application through one of the `MSIPConnectionObserver::ErrorOccured` callback functions. There are several overloaded `ErrorOccured` functions, each of them having the error code and reference(s) to the SIP Client API object(s) that encountered the error, as parameters.

When the SIP Client API passes an `ErrorOccured` event with a `CSIPTransactionBase` or `CSIPClientTransaction` parameter to the application, the transaction in question has entered the state `CTransactionBase::ETerminated`.

If the `ErrorOccured` event has `CSIPRefresh` as a parameter, the refresh has failed and its state is `CSIPRefresh::ETerminated`.

If the `ErrorOccured` event has `CSIPRegistration` as a parameter, it means that the registration context is no longer active (`IsContextActive() == EFalse`). If the registration was being refreshed, the refresh has also ended and its state is `CSIPRefresh::ETerminated`.

If the `ConnectionStateChanged` event is received with the value of `aState` parameter being either `CSIPConnection::EInactive` or `CSIPConnection::EUnavailable`, all the dialogs, registrations, and stand-alone transactions using that `CSIPConnection` have terminated. To avoid event flood in this kind of situation, the `ErrorOccured` events for the individual dialog, registration, and transaction objects are not sent to the application.

2.3.3 Memory overhead

After `CSIP` and a required amount of `CSIPConnection` instances have been created, most of the memory consumption occurs in the `CSIPMessageElements`, `CSIPRequestElements`, and `CSIPResponseElements` classes.

`CSIPTransactionBase` holds the most recent SIP response in a `CSIPResponseElements` object. When another response is received, the previous `CSIPResponseElements` object is deleted and a new `CSIPResponseElements` is created.

`CSIPClientTransaction` contains an instance of `CSIPRequestElements` for the duration it takes to transfer the SIP request to the server side of the SIP stack.

`CSIPServerTransaction` contains the SIP request as `CSIPRequestElements`.

Because the application owns the `CSIPClientTransaction` and `CSIPServerTransaction` objects, it is recommended to delete them as soon as the transaction is not needed anymore to conserve memory. A non-INVITE `CSIPClientTransaction` can be deleted when it has received a final response or the `ErrorOccured` event. A non-INVITE `CSIPServerTransaction` can be deleted after the application has sent a final response.

The INVITE-related transactions behave differently because of the presence of an ACK. Also, they should be deleted if an `ErrorOccured` event is received. When the application sends a 2xx response to an INVITE-related `CSIPServerTransaction`, it will enter the state `CSIPTransactionBase::ETerminated` and can be deleted at this point. The ACK will be received with a new `CSIPServerTransaction` instance anyway. But when the application has sent an INVITE request and the returned `CSIPClientTransaction` has received a 2xx response, the transaction enters the state `CSIPTransactionBase::ETerminated`. The associated `CSIPClientTransaction` should not be deleted yet since it will be needed for sending the ACK and also to receive the `MSIPConnectionObserver::InviteCompleted` event. So an INVITE-related `CSIPClientTransaction` should be deleted only after a `MSIPConnectionObserver::InviteCompleted` or a `MSIPConnectionObserver::ErrorOccured` event has been received, or if the state of the associated `CSIPConnection` goes to `CSIPConnection::EInactive` or `CSIPConnection::EUnavailable`.

2.3.4 Extensions to the API

There are no possible extensions to this API.

2.4 Example

The SIP Client API is initiated by creating the CSIP and CSIPConnection objects.

```
iSIP = CSIP::NewL(KAppUid, *iMySIPObserver);
iConnection = CSIPConnection::NewL(*iSIP, KIapIdentifier,
                                   *iMyConnectionObserver);
```

In order to publish a user's SIP contact address, a SIP registration is created. This requires an AOR, a contact to be registered; in this example, a CSIPRefresh is also created to cause the registration to be automatically refreshed by the SIP stack.

```
CSIPHostPort* hp = CSIPHostPort::NewLC(_L8("remote.host"));
CSIPURI* uri = CSIPURI::NewL(hp);
CleanupStack::Pop(hp);
CleanupStack::PushL(uri);
CSIPAddress* addr = CSIPAddress::NewL(uri);
CleanupStack::Pop(uri);
CleanupStack::PushL(addr);
CSIPToHeader* aor = CSIPToHeader::NewL(addr);
CleanupStack::Pop(addr);
CleanupStack::PushL(aor);

hp = CSIPHostPort::NewLC(_L8("users.contact"));
uri = CSIPURI::NewL(hp);
CleanupStack::Pop(hp);
CleanupStack::PushL(uri);
addr = CSIPAddress::NewL(uri);
CleanupStack::Pop(uri);
CleanupStack::PushL(addr);
CSIPContactHeader* contact = CSIPContactHeader::NewL(addr);
CleanupStack::Pop(addr);
CleanupStack::PushL(contact);

CSIPRefresh* refresh = CSIPRefresh::NewLC();

iRegistration = CSIPRegistration::NewL(*iConnection, aor, contact, refresh);

CleanupStack::Pop(3); //refresh, contact, aor
```

Once CSIPRegistration has been created, a REGISTER transaction can be initiated.

```
iRegisterTransaction = iRegistration->RegisterL();
```

When a 2xx response is received, the registration is active. Because a final response was received, the transaction can be deleted.

```
void CMyConnectionObserver::IncomingResponse(CSIPClientTransaction& aTransaction,
                                             CSIPRegistration& aRegistration)
{
    if (aRegistration.IsContextActive())
    {
        // Registration has succeeded
    }

    if (aTransaction.ResponseElements()->StatusCode() >= 200)
    {
        // Final response received, transaction is no longer needed

        if (aTransaction == *iRegisterTransaction)
        {
            delete iRegisterTransaction;
            iRegisterTransaction = NULL;
        }
    }
}
```

An INVITE request is received from the network. In this example, it is received through a different IAP than the one for which there already exists a CSIPConnection. A CSIPConnection instance is also created for this IAP.

```

void CMYsipObserver::IncomingRequest(TUint32 aIapId,
                                     CSIPServerTransaction* aTransaction)
{
    // Create an observer for the new CSIPConnection
    TRAPD(err, iMyOtherConnectionObserver = CMYConnectionObserver::NewL());
    // Handle leave condition

    // Create a new connection for the IAP
    TRAPD(err, iOtherConnection = CSIPConnection::NewL(*iSIP, aIapId,
                                                       *iMyOtherConnectionObserver));
    // Handle leave condition

    // Application owns the transaction
    iInviteServerTransaction = aTransaction;
}

```

Because the received request is an INVITE, and the application intends to send a response (101-299) that will create a SIP dialog, a CSIPInviteDialogAssoc needs to be created before sending the response.

```

iInviteAssoc = CSIPInviteDialogAssoc::NewL(*iInviteServerTransaction);
CSIPResponseElements* response = CSIPResponseElements::NewLC(180,
                                                             _L8("Ringing"));

iInviteServerTransaction->SendResponseL(response);
CleanupStack::Pop(response);

```

When the user has accepted the INVITE, a 200 response is sent using the same transaction object. Since sending a 2xx response to an INVITE causes the transaction to enter the CSIPTransactionBase::ETerminated state, the transaction can be deleted.

```

CSIPResponseElements* response = CSIPResponseElements::NewLC(200, _L8("OK"));

iInviteServerTransaction->SendResponseL(response);
CleanupStack::Pop(response);

delete iInviteServerTransaction;
iInviteServerTransaction = NULL;

```

When the ACK request arrives, the SIP Client API creates a new CSIPServerTransaction and passes the transaction object to the application using the MSIPConnectionObserver of the IAP through which the ACK was received. No response is sent to an ACK, so the transaction object can be deleted.

```

void CMYConnectionObserver::IncomingRequest(CSIPServerTransaction* aTransaction,
                                             CSIPDialog& aDialog)
{
    if (aTransaction.Type() == CSIPTransactionBase::EACK)
    {
        // Might inspect the Content part of the ACK for SDP etc.

        // No response will be sent to an ACK
        delete aTransaction;
    }
    else
    {
        // Handle other requests
    }
}

```

2.5 Detailed Description

2.5.1 CSIP

This class provides the connection to the shared SIP server and functions for querying the supported SIP security mechanisms and the support of the signaling compression (SigComp). The application can create only one instance of this class (a singleton class).

2.5.1.1 NewL

Function	<code>static CSIP* NewL(const TUid& aUid, MSIPObserver& aObserver);</code>
Description	Creates an instance of CSIP and connects to a shared server. If the server is not currently running, it will be started now.
Preconditions	None
Parameter	<code>const TUid& aUid</code> : A unique ID that identifies an application. <code>MSIPObserver& aObserver</code> : Observer for receiving asynchronous events.
Return value	<code>CSIP*</code> : A new instance of CSIP.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.1.2 NewLC

Function	<code>static CSIP* NewLC(const TUid& aUid, MSIPObserver& aObserver);</code>
Description	Creates an instance of CSIP and adds the pointer to the cleanup stack. Connects to a shared server. If the server is not currently running, it will be started now.
Preconditions	None
Parameter	<code>const TUid& aUid</code> : A unique ID that identifies an application. <code>MSIPObserver& aObserver</code> : Observer for asynchronous event.
Return value	<code>CSIP*</code> : A new instance of CSIP.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.1.3 ~CSIP

Function	<code>~CSIP();</code>
Description	Closes the connection to the server and deletes the resources of CSIP.
Preconditions	None
Parameter	-
Return value	-

Postconditions	None
Exceptions	-

2.5.1.4 SupportedSecurityMechanismsL

Function	CDesC8Array* SupportedSecurityMechanismsL() const;
Description	Gets all supported security mechanisms.
Preconditions	None
Parameter	-
Return value	CDesC8Array*: Supported SIP security mechanisms names (for example, digest); the ownership is transferred.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.1.5 IsSigCompSupportedL

Function	TBool IsSigCompSupportedL() const;
Description	Checks if the signaling compression (SigComp) is supported.
Preconditions	None
Parameter	-
Return value	TBool: ETrue if supported, otherwise EFalse.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.1.6 Connection

Function	CSIPConnection* Connection(TUint32 aIapId) const;
Description	Checks if the user has a connection created for given IAP-ID.
Preconditions	None
Parameter	TUint32 aIapId: IAP identifier.
Return value	CSIPConnection*: SIP connection or 0-pointer if connection does not exist. Ownership is not transferred.
Postconditions	None
Exceptions	-

2.5.2 CSIPClientTransaction

This class is used for managing SIP client transactions. It provides services for getting information of a received SIP response. Some client transactions can also be canceled. The application cannot instantiate this class.

2.5.2.1 ~CSIPClientTransaction

Function	~CSIPClientTransaction();
Description	Releases the resources of CSIPClientTransaction.
Preconditions	None
Parameter	-
Return value	-
Postconditions	None
Exceptions	-

2.5.2.2 ResponseElements

Function	const CSIPResponseElements* ResponseElements() const;
Description	Gets response elements.
Preconditions	None
Parameter	-
Return value	const CSIPResponseElements*: Response elements of the most recent response. Ownership isn't transferred.
Postconditions	None
Exceptions	-

2.5.2.3 Cancell

Function	CSIPClientTransaction* Cancell();
Description	Cancels the client transaction, that is, creates a CANCEL request and sends it to the remote UA.
Preconditions	State() == CSIPTransactionBase::EProceeding CancelAllowed() == ETrue CSIPConnection::State() == CSIPConnection::EActive
Parameter	-
Return value	CSIPClientTransaction*: SIP CANCEL transaction, ownership is transferred.
Postconditions	None
Exceptions	The function leaves with CSIP::EInvalidTransactionState if CancelAllowed() == EFalse or State() != EProceeding or if the transaction is not associated to a dialog. The function leaves with CSIP::EResourceNotAvailable if a required SIP Client API object has been deleted.

2.5.2.4 Refresh

Function	<code>const CSIPRefresh* Refresh() const;</code>
Description	Gets the associated refresh with the transaction. If the refresh is associated with the transaction, the transaction will be refreshed at a defined interval.
Preconditions	None
Parameter	-
Return value	<code>const CSIPRefresh*</code> : Associated refresh or 0-pointer if there's no associated refresh. Ownership isn't transferred.
Postconditions	None
Exceptions	-

2.5.2.5 CancelAllowed

Function	<code>TBool CancelAllowed() const;</code>
Description	Checks if the client transaction is such that it can be cancelled. This does not check whether the transaction is currently in such a state that the canceling can be done now.
Preconditions	None
Parameter	-
Return value	<code>TBool</code> : <code>ETrue</code> if client transaction can be cancelled, otherwise <code>EFa lse</code> .
Postconditions	None
Exceptions	-

2.5.3 CSIPConnection

The `CSIPConnection` class provides a way to monitor the connection's state, asking for the IAP identifier and sending SIP requests outside dialog associations.

2.5.3.1 NewL

Function	<code>static CSIPConnection* NewL(CSIP& aSIP, TUint32 aIapId, MSIPConnectionObserver& aObserver);</code>
Description	Creates an instance of <code>CSIPConnection</code> . Creates also resources on a shared server.
Preconditions	Application must not have a <code>CSIPConnection</code> instance for the same IAP identifier.
Parameter	<code>CSIP& aSIP</code> : Handle to the SIP server. <code>TUint32 aIapId</code> : IAP identifier. <code>MSIPConnectionObserver& aObserver</code> : Observer for receiving asynchronous events on this connection.
Return value	<code>CSIPConnection*</code> : A new instance of <code>CSIPConnection</code> .

Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.3.2 NewLC

Function	<code>static CSIPConnection* NewLC(CSIP& aSIP, TUint32 aIapId, MSIPConnectionObserver& aobserver);</code>
Description	Creates an instance of CSIPConnection and adds the pointer to the cleanup stack. Creates also resources on a shared server.
Preconditions	Application must not have a CSIPConnection instance for the same IAP identifier.
Parameter	CSIP& aSIP: Handle to the SIP server. TUint32 aIapId: IAP identifier. MSIPConnectionObserver& aobserver: Observer for receiving asynchronous events on this connection.
Return value	CSIPConnection*: A new instance of CSIPConnection.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.3.3 ~CSIPConnection

Function	<code>~CSIPConnection();</code>
Description	Closes the subsession formed with the server and deletes the resources of CSIPConnection.
Preconditions	None
Parameter	-
Return value	-
Postconditions	None
Exceptions	-

2.5.3.4 State

Function	<code>TState State() const;</code>
Description	Gets the current state of the connection.
Preconditions	None
Parameter	-
Return value	TState: Current connection state.
Postconditions	None
Exceptions	-

2.5.3.5 *SendRequestL*

Function	<code>CSIPClientTransaction* SendRequestL(CSIPRequestElements* aElements, const MSIPRegistrationContext& aContext);</code>
Description	Creates SIP request and sends it to the given destination. This provokes the non-invite state machine. This function must be used if the user has created the registration binding.
Preconditions	<code>State() == CSIPConnection::EActive</code> <code>aContext.IsContextActive() == ETrue</code> <code>aElements.Method() != "REGISTER" "INVITE" "BYE" "CANCEL" "ACK" "SUBSCRIBE" "NOTIFY" "UPDATE" "PRACK"</code>
Parameter	<code>CSIPRequestElements* aElements</code> : Contains local and remote addresses, optional SIP message headers, and body. The ownership is transferred. <code>const MSIPRegistrationContext& aContext</code> : Defines outbound proxy to be used. If the user does not provide from-address it will be constructed using user's AOR given during the registration.
Return value	<code>CSIPClientTransaction*</code> : SIP client transaction, ownership is transferred.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.3.6 *SendRequestL*

Function	<code>CSIPClientTransaction* SendRequestL(CSIPRequestElements* aElements, const MSIPRegistrationContext& aContext, CSIPRefresh& aRefresh);</code>
Description	Creates SIP request and sends it to the given destination. This provokes the non-invite state machine. This function must be used if the user has created the registration binding.
Preconditions	<code>State() == CSIPConnection::EActive</code> <code>aContext.IsContextActive() == ETrue</code> <code>aElements.Method() != "REGISTER" "INVITE" "BYE" "CANCEL" "ACK" "SUBSCRIBE" "NOTIFY" "UPDATE" "PRACK"</code>
Parameter	<code>CSIPRequestElements* aElements</code> : Contains local and remote addresses, optional SIP message headers, and body. The ownership is transferred. <code>const MSIPRegistrationContext & aContext</code> : Defines outbound proxy to be used. If the user does not provide from-address it will be constructed using user's AOR given during the registration. <code>CSIPRefresh& aRefresh</code> : Defines the refresh interval for this transaction.
Return value	<code>CSIPClientTransaction*</code> : SIP client transaction, ownership is transferred.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.3.7 *SendRequestL*

Function	<code>CSIPClientTransaction* SendRequestL(CSIPRequestElements* aElements);</code>
Description	Creates SIP request and sends it to the given destination. This invokes the non-invite state machine.
Preconditions	<code>State() == CSIPConnection::EActive</code> <code>aElements.FromHeader() != 0</code> <code>aElements.Method() != "REGISTER" "INVITE" "BYE" "CANCEL" "ACK" "SUBSCRIBE" "NOTIFY" "UPDATE" "PRACK"</code>
Parameter	<code>CSIPRequestElements* aElements</code> : Contains local and remote addresses, optional SIP message headers, and body. The ownership is transferred.
Return value	<code>CSIPClientTransaction*</code> : SIP client transaction, ownership is transferred.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.3.8 *SendRequestL*

Function	<code>CSIPClientTransaction* SendRequestL(CSIPRequestElements* aElements, CSIPRefresh& aRefresh);</code>
Description	Creates SIP request and sends it to the given destination. This invokes the non-invite state machine.
Preconditions	<code>State() == CSIPConnection::EActive</code> <code>Elements.FromHeader() != 0</code> <code>aElements.Method() != "REGISTER" "INVITE" "BYE" "CANCEL" "ACK" "SUBSCRIBE" "NOTIFY" "UPDATE" "PRACK"</code>
Parameter	<code>CSIPRequestElements* aElements</code> : Contains local and remote addresses, optional SIP message headers, and body. The ownership is transferred. <code>CSIPRefresh& aRefresh</code> : Indicates that transaction will be refreshed.
Return value	<code>CSIPClientTransaction*</code> : SIP client transaction, ownership is transferred.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.3.9 *FetchRegistrationsL*

Function	<code>CSIPClientTransaction* FetchRegistrationsL(CSIPRequestElements* aElements);</code>
Description	Creates SIP REGISTER request and sends it to the given destination.
Preconditions	<code>State() == Eactive</code> <code>aElements != 0</code> <code>aElements->Method().Length() == 0</code> <code>aElements.ToHeader().SIPAddress().URI().HasUser() == ETrue</code> <code>aElements</code> must not contain any Contact or Expires-headers

Parameter	aELeMents: Contains local and remote addresses, optional SIP message headers, and body. User must not set any expires values to aELeMents. The ownership is transferred.
Return value	CSIPClientTransaction*: SIP REGISTER client transaction, ownership is transferred.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.3.10 IapId

Function	TUint32 IapId() const;
Description	Gets the IAP identifier used for this connection.
Preconditions	None
Parameter	-
Return value	TUint32: IAP identifier.
Postconditions	None
Exceptions	-

2.5.4 CSIPDialog

The CSIPDialog class is used for managing SIP dialogs. It provides services querying the dialog state, obtaining the dialog-related SIP headers and getting all the dialog associations. The application cannot instantiate or delete this class.

2.5.4.1 State

Function	CSIPDialog::TState State() const;
Description	Gets the dialog state.
Preconditions	None
Parameter	-
Return value	CSIPDialog::TState: Dialog's current state.
Postconditions	None
Exceptions	-

2.5.4.2 SIPDialogAssociations

Function	const RPointerArray<CSIPDialogAssocBase>& SIPDialogAssociations() const;
Description	Gets all the dialog associations.
Preconditions	None
Parameter	-

Return value	const RPointerArray<CSIPDialogAssocBase>&: All dialog associations. Ownership of the array or the CSIPDialogAssocBase items is not transferred.
Postconditions	None
Exceptions	-

2.5.4.3 RegistrationContext

Function	const MSIPRegistrationContext* RegistrationContext() const;
Description	Gets the used registration context for this dialog.
Preconditions	None
Parameter	-
Return value	const MSIPRegistrationContext*: Associated registration or 0-pointer otherwise. Ownership is not transferred.
Postconditions	None
Exceptions	-

2.5.4.4 IsAssociated

Function	Tbool IsAssociated(const CSIPDialogAssocBase& aDialogAssoc) const;
Description	Checks if the dialog association belongs to this dialog.
Preconditions	None
Parameter	const CSIPDialogAssocBase& aDialogAssoc: A dialog association.
Return value	TBool: ETrue if belongs, otherwise EFalse.
Postconditions	None
Exceptions	-

2.5.4.5 Connection

Function	CSIPConnection* Connection();
Description	Gets the SIP connection used for this dialog.
Preconditions	None
Parameter	-
Return value	CSIPConnection*: SIP connection used for the dialog, or 0-pointer if the connection has been deleted. Ownership isn't transferred.
Postconditions	None
Exceptions	-

2.5.4.6 Connection

Function	<code>const CSIPConnection* Connection() const;</code>
Description	Gets the SIP connection used for this dialog.
Preconditions	None
Parameter	-
Return value	CSIPConnection*: SIP connection used for the dialog, or 0-pointer if the connection has been deleted. Ownership isn't transferred.
Postconditions	None
Exceptions	-

2.5.4.7 FromHeader

Function	<code>const CSIPFromHeader& FromHeader() const;</code>
Description	Gets originator's address.
Preconditions	None
Parameter	-
Return value	const CSIPFromHeader&: Originator's address (From-header).
Postconditions	None
Exceptions	-

2.5.4.8 ToHeader

Function	<code>const CSIPToHeader& ToHeader() const;</code>
Description	Gets recipient's address.
Preconditions	None
Parameter	-
Return value	const CSIPToHeader&: Recipient's address (To-header).
Postconditions	None
Exceptions	-

2.5.4.9 RemoteURI

Function	<code>const CURIBase* RemoteURI() const;</code>
Description	Gets remote-URI used during dialog creation.
Preconditions	None
Parameter	-
Return value	const CURIBase*: Remote target URI or 0-pointer. Ownership is not transferred.

Postconditions	None
Exceptions	-

2.5.4.10 operator==

Function	TBool operator==(const CSIPDialog& aDialog) const;
Description	Compares this object to another CSIPDialog object.
Preconditions	None
Parameter	const CSIPDialog& aDialog: A dialog object.
Return value	TBool: ETrue if the objects are equal, otherwise EFalse.
Postconditions	None
Exceptions	-

2.5.5 CSIPDialogAssocBase

CSIPDialogAssocBase is a base class for SIP dialog associations. It provides services for getting SIP dialog, actual dialog association type, and for sending nontarget refresh requests within the dialog association.

2.5.5.1 Dialog

Function	const CSIPDialog& Dialog() const;
Description	Gets the dialog this dialog association belongs to.
Preconditions	None
Parameter	-
Return value	const CSIPDialog&: The associated dialog.
Postconditions	None
Exceptions	-

2.5.5.2 Dialog

Function	CSIPDialog& Dialog();
Description	Gets dialog this dialog association belongs to.
Preconditions	None
Parameter	-
Return value	CSIPDialog&: The associated dialog.
Postconditions	None
Exceptions	-

2.5.5.3 SendNonTargetRefreshRequestL

Function	<code>virtual CSIPClientTransaction* SendNonTargetRefreshRequestL(const TDesC8& aMethod, CSIPMessageElements* aElements);</code>
Description	Creates a SIP request and sends it to the remote target. This function should be used for sending SIP extension requests within dialog associations that do not cause SIP dialog associations to be created. In SIP terms, this request is a nontarget refresh request.
Preconditions	<code>SIPDialog().State() != CSIPDialog::EInit</code> <code>SIPDialog().State() != CSIPDialog::ETerminated</code> <code>Dialog().Connection().State() == CSIPConnection::EActive</code> <code>aMethod != "REGISTER" "INVITE" "BYE" "CANCEL" "ACK" "SUBSCRIBE" "NOTIFY" "UPDATE" "PRACK"</code>
Parameter	<code>const TDesC8& aMethod</code> : SIP request method. <code>CSIPMessageElements* aElements</code> : Contains optional SIP message headers and body. Ownership is transferred.
Return value	<code>CSIPClientTransaction*</code> : SIP extension client transaction. Ownership is transferred.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.5.4 Type

Function	<code>TUint Type() const;</code>
Description	Gets dialog association type. The dialog association type is defined in each concrete dialog association class.
Preconditions	None
Parameter	-
Return value	<code>TUint</code> : A unique value identifying the dialog association type.
Postconditions	None
Exceptions	-

2.5.6 CSIPInviteDialogAssoc

`CSIPInviteDialogAssoc` is the class for managing a SIP dialog association created with INVITE. It provides services for creating, using, and terminating a SIP INVITE dialog association. The user can have only one INVITE dialog association per dialog at a time.

2.5.6.1 NewL

Function	<code>static CSIPInviteDialogAssoc* NewL(CSIPDialog& aSIPDialog);</code>
Description	Creates an instance of <code>CSIPInviteDialogAssoc</code> .
Preconditions	None

Parameter	CSIPDialog& aDialog: The dialog to be associated with.
Return value	CSIPInviteDialogAssoc*: New instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.6.2 NewLC

Function	Static CSIPInviteDialogAssoc* NewLC(CSIPDialog& aSIPDialog);
Description	Creates an instance of CSIPInviteDialogAssoc and adds the pointer to the cleanup stack.
Preconditions	None
Parameter	CSIPDialog& aDialog: The dialog to be associated with.
Return value	CSIPInviteDialogAssoc*: New instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.6.3 NewL

Function	static CSIPInviteDialogAssoc* NewL(CSIPDialog& aDialog, CSIPServerTransaction& aTransaction);
Description	Creates an instance of CSIPInviteDialogAssoc. This variant should be used if the response to a received SIP request (aTransaction) will create a SIP dialog association.
Preconditions	aTransaction.Type() == CSIPTransactionBase::EINVITE aTransaction.State() == CSIPTransactionBase::EProceeding
Parameter	CSIPDialog& aDialog: The dialog to be associated with. CSIPServerTransaction& aTransaction: A SIP server transaction.
Return value	CSIPInviteDialogAssoc*: New instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.6.4 NewLC

Function	static CSIPInviteDialogAssoc* NewLC(CSIPDialog& aDialog, CSIPServerTransaction& aTransaction);
Description	Creates an instance of CSIPInviteDialogAssoc and adds the pointer to the cleanup stack. This variant should be used if the response to a received SIP request (aTransaction) will create a SIP dialog association.
Preconditions	aTransaction.Type() == CSIPTransactionBase::EINVITE aTransaction.State() == CSIPTransactionBase::EProceeding
Parameter	CSIPDialog& aDialog: The dialog to be associated with. CSIPServerTransaction& aTransaction: A SIP server transaction.

Return value	CSIPInviteDialogAssoc*: New instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.6.5 NewL

Function	<code>static CSIPInviteDialogAssoc* NewL(CSIPServerTransaction& aTransaction);</code>
Description	Creates an instance of CSIPInviteDialogAssoc. This variant should be used if the response to a received SIP request (aTransaction) will create a SIP dialog association. A new CSIPDialog will also be created.
Preconditions	<code>aTransaction.Type() == CSIPTransactionBase::EINVITE</code> <code>aTransaction.State() == CSIPTransactionBase::EProceeding</code>
Parameter	CSIPServerTransaction& aTransaction: A SIP server transaction.
Return value	CSIPInviteDialogAssoc*: New instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.6.6 NewLC

Function	<code>static CSIPInviteDialogAssoc* NewLC(CSIPServerTransaction& aTransaction);</code>
Description	Creates an instance of CSIPInviteDialogAssoc and adds the pointer to the cleanup stack. This variant should be used if the response to a received SIP request (aTransaction) will create a SIP dialog association. A new CSIPDialog will also be created.
Preconditions	<code>aTransaction.Type() == CSIPTransactionBase::EINVITE</code> <code>aTransaction.State() == CSIPTransactionBase::EProceeding</code>
Parameter	CSIPServerTransaction& aTransaction: A SIP server transaction.
Return value	CSIPInviteDialogAssoc*: New instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.6.7 NewL

Function	<code>Static CSIPInviteDialogAssoc* NewL(CSIPConnection& aConnection, CSIPFromHeader* aFrom, CSIPToHeader* aTo, CURIBase* aRemoteUri=0, CSIPContactHeader* aContact=0);</code>
Description	Creates an instance of CSIPInviteDialogAssoc. A new CSIPDialog will also be created.
Preconditions	<code>aFrom != 0</code> <code>aTo != 0</code> <code>aFrom->HasTag() == EFalse</code> <code>aTo->HasTag() == EFalse</code>

Parameter	CSIPConnection& aConnection: A SIP connection to be used with dialog association. CSIPFromHeader* aFrom: Originator's address; the ownership is transferred. CSIPToHeader* aTo: Recipient's address; the ownership is transferred. CURIBase* aRemoteURI: A remote target URI. If not defined, the recipient's address will be used for remote-URI construction; the ownership is transferred. CSIPContactHeader* aContact: A contact to be used in dialog creation. Must be given only if the user intends to redirect future requests; the ownership is transferred.
Return value	CSIPInviteDialogAssoc*: New instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.6.8 NewLC

Function	static CSIPInviteDialogAssoc* NewLC(CSIPConnection& aConnection, CSIPFromHeader* aFrom, CSIPToHeader* aTo, CURIBase* aRemoteUri=0, CSIPContactHeader* aContact=0);
Description	Creates an instance of CSIPInviteDialogAssoc and adds the pointer to the cleanup stack. A new CSIPDialog will also be created.
Preconditions	aFrom != 0 aTo != 0 aFrom->HasTag() == EFalse aTo->HasTag() == EFalse
Parameter	CSIPConnection& aConnection: A SIP connection to be used with dialog association. CSIPFromHeader* aFrom: Originator's address; the ownership is transferred. CSIPToHeader& aTo: Recipient's address; the ownership is transferred. CURIBase* aRemoteURI: A remote target URI. If not defined, the recipient's address will be used for remote-URI construction; the ownership is transferred. CSIPContactHeader* aContact: A contact to be used in dialog creation. Must be given only if user intends to redirect future requests; the ownership is transferred.
Return value	CSIPInviteDialogAssoc*: New instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.6.9 NewL

Function	<code>static CSIPInviteDialogAssoc* NewL(CSIPConnection& aConnection, CSIPToHeader* aTo, const MSIPRegistrationContext& aContext, CSIPFromHeader* aFrom = 0, CURIBase* aRemoteUri = 0, CSIPContactHeader* aContact = 0);</code>
Description	Creates an instance of CSIPInviteDialogAssoc. A new CSIPDialog will also be created.
Preconditions	<code>aTo != 0</code> <code>aFrom->HasTag() == EFalse</code> <code>aTo->HasTag() == EFalse</code> <code>aContext.IsActive() == Etrue</code>
Parameter	CSIPConnection& aConnection: A SIP connection to be used with dialog association. CSIPToHeader* aTo: Recipient's address; the ownership is transferred. const MSIPRegistrationContext& aContext: Used for selecting outbound proxy and recipient's address (AOR) and contact. CSIPFromHeader* aFrom: Originator's address. If not defined it will be constructed using registration context (user's AOR); the ownership is transferred. CURIBase* aRemoteURI: A remote target URI. If not defined, the recipient's address will be used for request-URI construction; the ownership is transferred. CSIPContactHeader* aContact: A contact to be used in dialog creation. Must be given only if user intends to redirect future requests; the ownership is transferred.
Return value	CSIPInviteDialogAssoc*: New instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.6.10 NewLC

Function	<code>static CSIPInviteDialogAssoc* NewLC(CSIPConnection& aConnection, CSIPToHeader* aTo, const MSIPRegistrationContext& aContext, CSIPFromHeader* aFrom = 0, CURIBase* aRemoteUri = 0, CSIPContactHeader* aContact = 0);</code>
Description	Creates an instance of CSIPInviteDialogAssoc and adds the pointer to the cleanup stack. A new CSIPDialog will also be created.
Preconditions	<code>aTo != 0</code> <code>aFrom->HasTag() == EFalse</code> <code>aTo->HasTag() == EFalse</code> <code>aContext.IsActive() == ETrue</code>

Parameter	CSIPConnection& aConnection: A SIP connection to be used with dialog association. CSIPToHeader* aTo: Recipient's address; the ownership is transferred. const MSIPRegistrationContext& aContext: Used for selecting outbound proxy and originator's address (AOR) and contact. CSIPFromHeader& aFrom: Originator's address. If not defined it will be constructed using registration context (user's AOR); the ownership is transferred. CURIBase& aRemoteURI: A remote target URI. If not defined, the recipient's address will be used for request-URI construction; the ownership is transferred. CSIPContactHeader* aContact: A contact to be used in dialog creation. Must be given only if user intends to redirect future requests; the ownership is transferred.
Return value	CSIPInviteDialogAssoc*: New instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.6.11 ~CSIPInviteDialogAssoc()

Function	~CSIPInviteDialogAssoc();
Description	Deletes the resources of CSIPInviteDialogAssoc.
Preconditions	None
Parameter	-
Return value	-
Postconditions	None
Exceptions	-

2.5.6.12 SendInviteL

Function	CSIPClientTransaction* SendInviteL(CSIPMessageElements* aElements=0);
Description	Creates INVITE and sends it to the remote target. A 101-199 or 2xx response will create actual SIP INVITE dialog association in case of first SIP request within this dialog association. Other INVITE requests are re-INVITES.
Preconditions	Dialog().Connection().State() == CSIPConnection::EActive Dialog().State() == CSIPDialog::EInit Dialog().State() == CSIPDialog::EConfirmed
Parameter	CSIPMessageElements& aElements: Contains user SIP headers and content. Ownership is transferred.
Return value	CSIPClientTransaction*: INVITE SIP transaction. Ownership is transferred.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.6.13 *SendPrackL*

Function	CSIPClientTransaction* SendPrackL(CSIPMessageElements* aElements=0);
Description	Creates PRACK and sends it to the remote target.
Preconditions	Dialog().Connection().State() == CSIPConnection::EActive Dialog().State() == CSIPDialog::EEarly Dialog().State() == CSIPDialog::EConfirmed aElements must not contain Contact headers
Parameter	CSIPMessageElements* aElements: Contains user SIP headers and content. Ownership is transferred.
Return value	CSIPClientTransaction*: PRACK SIP transaction. Ownership is transferred.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.6.14 *SendUpdateL*

Function	CSIPClientTransaction* SendUpdateL(CSIPMessageElements* aElements=0);
Description	Creates UPDATE and sends it to the remote target.
Preconditions	Dialog().Connection().State() == CSIPConnection::EActive Dialog().State() == CSIPDialog::EEarly Dialog().State() == CSIPDialog::EConfirmed
Parameter	CSIPMessageElements* aElements: Contains user SIP headers and content. Ownership is transferred.
Return value	CSIPClientTransaction*: UPDATE SIP transaction. Ownership is transferred.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.6.15 *SendAckL*

Function	void SendAckL(const CSIPClientTransaction& aTransaction, CSIPMessageElements* aElements=0);
Description	Creates SIP ACK request and sends it to the remote target.
Preconditions	Dialog().Connection().State() == CSIPConnection::EActive aSIPTransaction.Type() == CSIPTransactionBase::EINVITE
Parameter	const CSIPClientTransaction& aTransaction: The SIP INVITE client transaction to acknowledge. CSIPMessageElements* aElements: Optional SIP message headers and body. Ownership is transferred.
Return value	-

Postconditions	None
Exceptions	In case of an error, the function leaves. The function leaves with <code>KErrArgument</code> if <code>aTransaction.Type() != CSIPTransactionBase::EINVITE</code>

2.5.6.16 SendByeL

Function	<code>CSIPClientTransaction* SendByeL(CSIPMessageElements* aElements=0);</code>
Description	Creates SIP BYE request and sends it to the remote target.
Preconditions	<code>Dialog().Connection().State() == CSIPConnection::EActive</code>
Parameter	<code>CSIPMessageElements* aElements</code> : Contains user SIP headers and content. Ownership is transferred.
Return value	<code>CSIPClientTransaction*</code> : SIP BYE transaction. Ownership is transferred.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.7 CSIPMessageElements

`CSIPMessageElements` is the class for creating and manipulating the optional elements of a SIP message. `CSIPMessageElements` provides functions for setting and getting optional elements in a SIP message. Optional elements include user SIP message headers, content, and content type.

The following headers are not allowed to be manipulated with `CSIPMessageElements`:
`CSIPHeaderBase::FullName() != ("Authentication-Info" || "Call-Id" || "CSeq" || "From" || "Max-Forwards" || "Min-Expires" || "Record-Route" || "Security-Verify" || "Service-Route" || "To" || "Via")`.

2.5.7.1 NewL

Function	<code>static CSIPMessageElements* NewL();</code>
Description	Creates an instance of <code>CSIPMessageElements</code> .
Preconditions	None
Parameter	-
Return value	<code>CSIPMessageElements*</code> : New instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.7.2 NewLC

Function	<code>static CSIPMessageElements* NewLC();</code>
Description	Creates an instance of <code>CSIPMessageElements</code> and adds the pointer to the cleanup stack.
Preconditions	None

Parameter	-
Return value	CSIPMessageElements*: New instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.7.3 ~CSIPMessageElements

Function	<code>virtual ~CSIPMessageElements();</code>
Description	Deletes the resources of CSIPMessageElements.
Preconditions	None
Parameter	-
Return value	-
Postconditions	None
Exceptions	-

2.5.7.4 SetUserHeadersL

Function	<code>void SetUserHeadersL(RPointerArray<CSIPHeaderBase>& aHeaders);</code>
Description	Sets an array of user headers, that is, headers that the user is allowed to manipulate to a SIP message. An empty array resets the user headers. Note that the Content-Type header must be set using SetContentL.
Preconditions	None
Parameter	RPointerArray<CSIPHeaderBase>& aHeaders: An array of SIP headers. The ownership of objects in the array is transferred.
Return value	-
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.7.5 UserHeaders

Function	<code>const RPointerArray<CSIPHeaderBase>& UserHeaders() const;</code>
Description	Gets all user SIP headers this class contains.
Preconditions	None
Parameter	-
Return value	const RPointerArray<CSIPHeaderBase>& SIP headers. Ownership is not transferred.
Postconditions	None
Exceptions	-

2.5.7.6 *SetContentL*

Function	<code>void SetContentL(HBufC8* aContent, CSIPContentTypeHeader* aContentType);</code>
Description	Sets the SIP message content and its type.
Preconditions	<code>aContent != 0</code> <code>aContent->Length() > 0</code> <code>aContentType != 0</code>
Parameter	<code>HBufC8* aContent</code> : The content of a SIP message, the ownership is transferred. <code>CSIPContentTypeHeader* aContentType</code> : The SIP message content type, the ownership is transferred.
Return value	-
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.7.7 *Content*

Function	<code>const TDesC8& Content() const;</code>
Description	Gets the SIP message content.
Preconditions	None
Parameter	-
Return value	<code>const TDesC8&</code> : SIP message content. Ownership is not transferred. If content does not exist, an empty descriptor is returned.
Postconditions	None
Exceptions	-

2.5.7.8 *ContentType*

Function	<code>const CSIPContentTypeHeader* ContentType() const;</code>
Description	Gets the content type.
Preconditions	None
Parameter	-
Return value	<code>const CSIPContentTypeHeader*</code> : Content-Type-header or a 0-pointer if not present. Ownership is not transferred.
Postconditions	None
Exceptions	-

2.5.7.9 ExtractContent

Function	HBufC8* ExtractContent();
Description	Removes the SIP message content. Also the Content-Type header is destroyed.
Preconditions	None
Parameter	-
Return value	HBufC8*: SIP message content. The ownership is transferred.
Postconditions	None
Exceptions	-

2.5.8 CSIPRefresh

This class is used for managing SIP refreshes. It provides functions for getting the associated SIP transaction and the state of the refresh. This class also provides functions for updating and terminating stand-alone refreshes.

To update and terminate a SUBSCRIBE-related refresh, the CSIPSubscribeDialogAssoc class should be used instead.

To update and terminate a REGISTER-related refresh, the CSIPRegistration class should be used instead.

2.5.8.1 NewL

Function	static CSIPRefresh* NewL();
Description	Creates an instance of CSIPRefresh.
Preconditions	None
Parameter	-
Return value	CSIPRefresh*: New instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.8.2 NewLC

Function	Static CSIPRefresh* NewLC();
Description	Creates an instance of CSIPRefresh and adds the pointer to the cleanup stack.
Preconditions	None
Parameter	-
Return value	CSIPRefresh*: New instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.8.3 ~CSIPRefresh

Function	<code>~CSIPRefresh();</code>
Description	Deletes the resources of CSIPRefresh.
Preconditions	None
Parameter	-
Return value	-
Postconditions	None
Exceptions	-

2.5.8.4 State

Function	<code>CSIPRefresh::TState State() const;</code>
Description	Gets the state of the refresh.
Preconditions	None
Parameter	-
Return value	<code>CSIPRefresh::TState</code> : State of the refresh.
Postconditions	None
Exceptions	-

2.5.8.5 SIPTransaction

Function	<code>const CSIPClientTransaction* SIPTransaction() const;</code>
Description	Gets the associated SIP transaction with this refresh.
Preconditions	None
Parameter	-
Return value	<code>const CSIPClientTransaction*</code> : Associated SIP transaction or 0-pointer. Ownership is not transferred.
Postconditions	None
Exceptions	-

2.5.8.6 TerminateL

Function	<code>CSIPClientTransaction* TerminateL(CSIPMessageElements* aElements=0);</code>
Description	Terminates the refresh by sending a SIP request to the remote destination. The new client transaction will be of the same type as the first transaction associated with this request.
Preconditions	<code>State() == CSIPRefresh::EActive</code>

Parameter	CSIPMessageElements* aElements: Contains optional SIP message headers and body. Ownership is transferred.
Return value	CSIPClientTransaction*: SIP client transaction. Ownership is transferred.
Postconditions	None
Exceptions	In case of an error, the function leaves. The function leaves with KErrNotFound if the CSIPRefresh is no longer associated with anything. The function leaves with CSIP::EInvalidRegistrationState if trying to terminate a REGISTER refresh. Use CSIPRegistration class instead to do that. The function leaves with CSIP::EInvalidDialogState if trying to terminate a SUBSCRIBE refresh. Use CSIPSubscribeDialogAssoc class instead to do that.

2.5.8.7 UpdateL

Function	CSIPClientTransaction* UpdateL(CSIPMessageElements* aElements=0);
Description	Updates the refresh by sending a SIP request to the remote destination. The new client transaction will be of the same type as the first transaction associated with this request.
Preconditions	State() == CSIPRefresh::Eactive
Parameter	CSIPMessageElements* aElements: Contains optional SIP message headers and body. Ownership is transferred.
Return value	CSIPClientTransaction*: SIP client transactions. Ownership is transferred.
Postconditions	None
Exceptions	In case of an error, the function leaves. The function leaves with KErrNotFound if the CSIPRefresh is no longer associated to anything. The function leaves with CSIP::EInvalidRegistrationState if trying to update a REGISTER refresh. Use CSIPRegistration class instead to do that. The function leaves with CSIP::EInvalidDialogState if trying to update a SUBSCRIBE refresh. Use CSIPSubscribeDialogAssoc class instead to do that.

2.5.8.8 operator==

Function	TBool operator==(const CSIPRefresh& aRefresh) const;
Description	Compares this object to another CSIPRefresh object.
Preconditions	None
Parameter	const CSIPRefresh& aRefresh: A CSIPRefresh object to compare.
Return value	TBool: ETrue if the objects are equal, otherwise EFalse.
Postconditions	None
Exceptions	-

2.5.9 CSIPRegistration

This is the class for managing SIP registrations. CSIPRegistration provides functions for creating and updating SIP registration bindings.

The application can also use the CSIPRegistration object after the registration context has gone inactive (IsContextActive() == EFalse) — for example, the application has registered with a refresh, and later the registration encounters an error, causing the refresh to end and context to be inactive. At this point, the application can re-initiate the registration with the RegisterL function. Since the application defined that the registration should be refreshed when it created CSIPRegistration with NewL/NewLC, the re-initiated registration will also be refreshed.

2.5.9.1 NewL

Function	static CSIPRegistration* NewL(CSIPConnection& aConnection, CSIPToHeader* aAor, CSIPContactHeader* aContact, CSIPRefresh* aRefresh=0, CSIPRouteHeader* aOutboundProxy=0, CSIPURI* aRemoteUri=0, CSIPFromHeader* aFrom=0);
Description	Creates an instance of CSIPRegistration.
Preconditions	aAor != 0 aContact != 0 "Expires" parameter in aContact must be >= KSIPMinExpirationValue
Parameter	CSIPConnection& aConnection: A SIP connection to be used. CSIPToHeader* aAor: Contains user's address-of-record to register; the ownership is transferred. CSIPContactHeader* aContact: Contact to be registered. The "expires" parameter indicates how long the client would like the binding to be valid. The "expires" parameter value must be >= KSIPMinExpirationValue; the ownership is transferred. CSIPRefresh* aRefresh: If set, created binding will be refreshed; the ownership is transferred. CSIPRouteHeader* aOutboundProxy: An outbound proxy address; the ownership is transferred. CSIPURI* aRemoteUri: If set, will be used in request-URI construction; the ownership is transferred. CSIPFromHeader* aFrom: From header, the ownership is transferred.
Return value	CSIPRegistration*: New instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.9.2 *NewLC*

Function	<code>static CSIPRegistration* NewLC(CSIPConnection& aConnection, CSIPToHeader* aAor, CSIPContactHeader* aContact, CSIPRefresh* aRefresh=0, CSIPRouteHeader* aOutboundProxy=0, CSIPURI* aRemoteUri=0, CSIPFromHeader* aFrom=0);</code>
Description	Creates an instance of CSIPRegistration and adds the pointer to the cleanup stack.
Preconditions	aAor != 0 aContact != 0 "Expires" parameter in aContact must be >= KSIPMinExpirationValue
Parameter	CSIPConnection& aConnection: A SIP connection to be used. CSIPToHeader* aAor: Contains user's address-of-record to register; the ownership is transferred. CSIPContactHeader* aContact: The contact to be registered. The "expires" parameter indicates how long the client would like the binding to be valid. The "expires" parameter value must be >= KSIPMinExpirationValue; the ownership is transferred. CSIPRefresh* aRefresh: If set, created binding will be refreshed; the ownership is transferred. CSIPRouteHeader* aOutboundProxy: An outbound proxy address; the ownership is transferred. CSIPURI* aRemoteUri: If set, will be used in request-URI construction; the ownership is transferred. CSIPFromHeader* aFrom: From header, the ownership is transferred.
Return value	CSIPRegistration*: New instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.9.3 *~CSIPRegistration*

Function	<code>~CSIPRegistration();</code>
Description	Deletes the resources of CSIPRegistration. This does not cause a (de)REGISTER request being sent to network.
Preconditions	None
Parameter	-
Return value	-
Postconditions	None
Exceptions	-

2.5.9.4 RegisterL

Function	CSIPClientTransaction* RegisterL(CSIPMessageElements* aElements=0);
Description	Creates a SIP REGISTER request, sends it to the given destination. A received 2xx response will create exactly one registration binding.
Preconditions	Connection().State() == CSIPConnection::EActive
Parameter	CSIPMessageElements* aElements: Contains user SIP headers and content; the ownership is taken.
Return value	CSIPClientTransaction*: SIP REGISTER transaction; the ownership is transferred.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.9.5 DeregisterL

Function	CSIPClientTransaction* DeregisterL(CSIPMessageElements* aElements=0);
Description	Creates a SIP (de)REGISTER request, sends it to the given destination. This function will remove registration binding. Any SIP response (2xx - 6xx) to this request removes the binding. Note that deregistration can be done, then a 2xx is received to the initial REGISTER transaction and there are no ongoing registration updates.
Preconditions	Connection().State() == CSIPConnection::EActive
Parameter	CSIPMessageElements* aElements: Contains user SIP headers and content; the ownership is transferred. If set, user SIP headers must not contain any Contact-header or Expires-header.
Return value	CSIPClientTransaction*: SIP REGISTER transaction; the ownership is transferred.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.9.6 UpdateL

Function	CSIPClientTransaction* UpdateL(CSIPMessageElements* aElements);
Description	Updates the registration binding. Note that update can be done, then the 2xx is received to the REGISTER transaction that relates to the initial registration or to update.
Preconditions	aElements != 0 Connection().State() == CSIPConnection::EActive
Parameter	CSIPMessageElements* aElements: Contains user SIP header and content, must not contain any Contact-header or Expires-header; the ownership is transferred.
Return value	CSIPClientTransaction*: SIP REGISTER transaction; the ownership is transferred.

Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.9.7 *UpdateL*

Function	<code>CSIPClientTransaction* UpdateL(TUint aExpirationValue, CSIPMessageElements* aElements=0);</code>
Description	Updates the registration binding. Note that update can be done, then the 2xx is received to the REGISTER transaction that relates to the initial registration or to update.
Preconditions	<code>aExpirationValue >= KSIPMinExpirationValue</code> <code>Connection().State() == CSIPConnection::EActive</code>
Parameter	<code>TUint aExpirationValue</code> : A new expiration value in seconds. <code>CSIPMessageElements* aElements</code> : Contains user SIP headers and content; must not contain any Contact-header or Expires-header; the ownership is transferred.
Return value	<code>CSIPClientTransaction*</code> : SIP REGISTER transaction; the ownership is transferred.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.9.8 *Connection*

Function	<code>CSIPConnection* Connection();</code>
Description	Gets the SIP connection used for the registration.
Preconditions	None
Parameter	-
Return value	<code>CSIPConnection*</code> : SIP connection used for the registration, or 0-pointer if the connection has been deleted. Ownership is not transferred.
Postconditions	None
Exceptions	-

2.5.9.9 *Connection*

Function	<code>const CSIPConnection* Connection();</code>
Description	Gets the SIP connection used for the registration.
Preconditions	None
Parameter	-
Return value	<code>CSIPConnection*</code> : SIP connection used for the registration, or 0-pointer if the connection has been deleted. Ownership is not transferred.

Postconditions	None
Exceptions	-

2.5.9.10 SetOutboundProxyL

Function	<code>void SetOutboundProxyL(CSIPRouteHeader* aOutboundProxy);</code>
Description	Sets/resets the outbound proxy. If a 0-pointer is given, the outbound proxy will be reset.
Preconditions	-
Parameter	CSIPRouteHeader* aOutboundProxy: An outbound proxy. Ownership is transferred.
Return value	-
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.9.11 Aor

Function	<code>const CSIPToHeader& Aor() const;</code>
Description	Gets the address-of-record (AOR) used for creating binding.
Preconditions	None
Parameter	-
Return value	const CSIPToHeader&: Address-of-record (AOR).
Postconditions	None
Exceptions	-

2.5.9.12 ContactHeader

Function	<code>const CSIPContactHeader& ContactHeader() const;</code>
Description	Gets Contact header associated with the registration binding.
Preconditions	None
Parameter	-
Return value	const CSIPContactHeader&: Contact header.
Postconditions	None
Exceptions	-

2.5.9.13 OutboundProxy

Function	<code>const CSIPRouteHeader* OutboundProxy() const;</code>
Description	Gets the used outbound proxy.
Preconditions	None
Parameter	-
Return value	<code>const CSIPRouteHeader*</code> : Outbound proxy or 0-pointer if not set. Ownership is not transferred.
Postconditions	None
Exceptions	-

2.5.9.14 SIPRefresh

Function	<code>const CSIPRefresh* SIPRefresh() const;</code>
Description	Gets associated refresh in case the user has requested the refresh of the registration. Note that client can only use the const functions of the returned CSIPRefresh, so, for example, the <code>TerminateL</code> and <code>UpdateL</code> functions are not available for use.
Preconditions	None
Parameter	-
Return value	<code>const CSIPRefresh*</code> : Associated refresh or 0-pointer if the user has not requested a refresh. Ownership is not transferred.
Postconditions	None
Exceptions	-

2.5.9.15 operator==

Function	<code>TBool operator==(const CSIPRegistration& aRegistration) const;</code>
Description	Compares this object to another CSIPRegistration object.
Preconditions	None
Parameter	<code>const CSIPRegistration& aRegistration</code> : CSIPRegistration object to compare.
Return value	<code>TBool</code> : <code>ETrue</code> if the objects are equal, otherwise <code>EFalse</code> .
Postconditions	None
Exceptions	-

2.5.9.16 IsContextActive

Function	<code>TBool IsContextActive() const;</code>
Description	Checks if the registration context can be used for creating SIP messages/dialogs.

Preconditions	None
Parameter	-
Return value	TBool: ETrue if the registration binding exists, otherwise EFalse.
Postconditions	None
Exceptions	-

2.5.9.17 ContextId

Function	TUint32 ContextId() const;
Description	Gets the context ID. This function is for internal use only.
Preconditions	None
Parameter	-
Return value	TUint32: Context ID
Postconditions	None
Exceptions	-

2.5.10 CSIPRequestElements

This class provides functions for creation and manipulation of the originator's and recipient's addresses in a SIP request. It also provides functions for manipulation of the SIP method for unknown SIP requests.

2.5.10.1 NewL

Function	static CSIPRequestElements* NewL(CSIPToHeader* aToHeader);
Description	Creates an instance of CSIPRequestElements.
Preconditions	aToHeader != 0
Parameter	CSIPToHeader* aToHeader: The To-header. The ownership is transferred.
Return value	CSIPRequestElements*: New instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.10.2 NewLC

Function	static CSIPRequestElements* NewLC(CSIPToHeader* aToHeader);
Description	Creates an instance of CSIPRequestElements and adds the pointer to the cleanup stack.
Preconditions	aToHeader != 0
Parameter	CSIPToHeader* aToHeader: The To-header. The ownership is transferred.

Return value	CSIPRequestElements*: New instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.10.3 ~CSIPRequestElements

Function	~CSIPRequestElements();
Description	Deletes the resources of CSIPRequestElements.
Preconditions	None
Parameter	-
Return value	-
Postconditions	None
Exceptions	-

2.5.10.4 SetToHeaderL

Function	void SetToHeaderL(CSIPToHeader* aTo);
Description	Sets the recipient's To-header.
Preconditions	aTo != 0 aTo->HasTag() == EFalse
Parameter	CSIPToHeader* aTo: A To-header to be set, the ownership is transferred.
Return value	-
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.10.5 ToHeader

Function	const CSIPToHeader* ToHeader() const;
Description	Gets the recipient's To-header.
Preconditions	None
Parameter	-
Return value	const CSIPToHeader*: To-header or a 0-pointer if not present. Ownership is not transferred.
Postconditions	None
Exceptions	-

2.5.10.6 SetFromHeaderL

Function	Void SetFromHeaderL(CSIPFromHeader* aFrom);
Description	Sets the originator's From-header.
Preconditions	aFrom != 0 aFrom->HasTag() == EFalse
Parameter	CSIPFromHeader* aFrom: A From-header to be set, the ownership is transferred.
Return value	-
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.10.7 FromHeader

Function	const CSIPFromHeader* FromHeader() const;
Description	Gets the originator's From-header.
Preconditions	None
Parameter	-
Return value	const CSIPFromHeader*: From-header or a 0-pointer if not present. Ownership is not transferred.
Postconditions	None
Exceptions	-

2.5.10.8 SetRemoteURL

Function	void SetRemoteURL(CURIBase* aURI);
Description	Sets the remote URI. If the URI is a SIP URI, no SIP URI headers are allowed.
Preconditions	aURI != 0 If aURI is a SIP URI, it must have a host part.
Parameter	CURIBase* aURI: The remote URI. The ownership is transferred.
Return value	-
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.10.9 RemoteURI

Function	const CURIBase* RemoteURI() const;
Description	Gets the remote URI.
Preconditions	None
Parameter	-

Return value	const CURIBase*: Remote URI or 0-pointer if not present. Ownership is not transferred.
Postconditions	None
Exceptions	-

2.5.10.10 *SetMethodL*

Function	void SetMethodL(const TDesC8& aMethod);
Description	Sets the SIP Method for a request.
Preconditions	aMethod.Length() > 0
Parameter	const TDesC8& aMethod: A SIP method name.
Return value	-
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.10.11 *Method*

Function	const TDesC8& Method() const;
Description	Gets the SIP method of the request.
Preconditions	None
Parameter	-
Return value	const TDesC8&: SIP method name or 0-length descriptor if the method is not defined.
Postconditions	None
Exceptions	-

2.5.10.12 *MessageElements*

Function	const CSIPMessageElements& MessageElements() const;
Description	Gets message elements (contains all SIP user headers and content).
Preconditions	None
Parameter	-
Return value	const CSIPMessageElements&: Message elements. Ownership is not transferred.
Postconditions	None
Exceptions	-

2.5.10.13 *MessageElements*

Function	<code>CSIPMessageElements& MessageElements();</code>
Description	Gets message elements (contains all SIP user headers and content). The response elements can be populated with SIP user headers and content using returned reference to the message elements.
Preconditions	None
Parameter	-
Return value	<code>CSIPMessageElements&</code> : Message elements. Ownership is not transferred.
Postconditions	None
Exceptions	-

2.5.11 *CSIPResponseElements*

This class provides services for creating and manipulating SIP responses. `CSIPResponseElements` is used for creating and manipulating SIP responses including status code, reason phrase, and optional elements such as user headers, content, and its type.

2.5.11.1 *NewL*

Function	<code>static CSIPResponseElements* NewL(TUint aStatusCode, const TDesC8& aReasonPhrase);</code>
Description	Creates an instance of <code>CSIPResponseElements</code> .
Preconditions	<code>aStatusCode > 100</code> <code>aStatusCode < 700</code>
Parameter	<code>TUint aStatusCode</code> : A known SIP response status code. Application cannot send a 100 response. <code>const TDesC8& aReasonPhrase</code> : A SIP response reason phrase.
Return value	<code>CSIPResponseElements*</code> : New instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.11.2 *NewLC*

Function	<code>static CSIPResponseElements* NewLC(TUint aStatusCode, const TDesC8& aReasonPhrase);</code>
Description	Creates an instance of <code>CSIPResponseElements</code> and adds the pointer to the cleanup stack.
Preconditions	<code>aStatusCode > 100</code> <code>aStatusCode < 700</code>
Parameter	<code>TUint aStatusCode</code> : A known SIP response status code. Application cannot send a 100 response. <code>const TDesC8& aReasonPhrase</code> : A SIP response reason phrase.
Return value	<code>CSIPResponseElements*</code> : New instance.

Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.11.3 ~CSIPResponseElements

Function	<code>~CSIPResponseElements();</code>
Description	Deletes the resources of CSIPResponseElements.
Preconditions	None
Parameter	-
Return value	-
Postconditions	None
Exceptions	-

2.5.11.4 SetStatusCodeL

Function	<code>void SetStatusCodeL(TUint aStatusCode);</code>
Description	Sets a SIP Response extension status code. It is not possible to set the status code to value 100.
Preconditions	<code>aStatusCode > 100</code> <code>aStatusCode < 700</code>
Parameter	TUint aStatusCode: Extension status code.
Return value	-
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.11.5 StatusCode

Function	<code>TUint StatusCode() const;</code>
Description	Gets the SIP Response status code.
Preconditions	None
Parameter	-
Return value	TUint: SIP Response status code.
Postconditions	None
Exceptions	-

2.5.11.6 *SetReasonPhraseL*

Function	<code>void SetReasonPhraseL(const TDesC8& aReasonPhrase);</code>
Description	Sets a SIP Response Reason Phrase.
Preconditions	None
Parameter	aReasonPhrase: A SIP response reason phrase.
Return value	-
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.11.7 *ReasonPhrase*

Function	<code>const TDesC8& ReasonPhrase() const;</code>
Description	Gets a SIP Response Reason Phrase.
Preconditions	None
Parameter	-
Return value	<code>const TDesC8&</code> : SIP Response reason phrase, or a 0-length descriptor if the reason phrase is not defined.
Postconditions	None
Exceptions	-

2.5.11.8 *FromHeader*

Function	<code>const CSIPFromHeader* FromHeader() const;</code>
Description	Gets the originator's From-header.
Preconditions	None
Parameter	-
Return value	<code>const CSIPFromHeader*</code> : A From-header or a 0-pointer if not present. Ownership is not transferred.
Postconditions	None
Exceptions	-

2.5.11.9 *ToHeader*

Function	<code>const CSIPToHeader* ToHeader() const;</code>
Description	Gets the recipient's To-header.
Preconditions	None
Parameter	-

Return value	const CSIPToHeader*: A To-header or a 0-pointer if not present. Ownership is not transferred.
Postconditions	None
Exceptions	-

2.5.11.10 CSeqHeader

Function	const CSIPCSeqHeader* CSeqHeader() const;
Description	Gets CSeq-header.
Preconditions	None
Parameter	-
Return value	const CSIPCSeqHeader*: A CSeq-header or a 0-pointer if not present. Ownership is not transferred.
Postconditions	None
Exceptions	-

2.5.11.11 MessageElements

Function	const CSIPMessageElements& MessageElements() const;
Description	Gets message elements (contains all SIP user headers and content).
Preconditions	None
Parameter	-
Return value	const CSIPMessageElements&: Message elements. Ownership is not transferred.
Postconditions	None
Exceptions	-

2.5.11.12 MessageElements

Function	CSIPMessageElements& MessageElements();
Description	Gets message elements (contains all SIP user headers and content). The response elements can be populated with SIP user headers and content using returned reference to the message elements.
Preconditions	None
Parameter	-
Return value	CSIPMessageElements&: Message elements. Ownership is not transferred.
Postconditions	None
Exceptions	-

2.5.12 CSIPServerTransaction

This class is for managing SIP server transactions. It provides services for creating, ending, and getting SIP transaction parameters. The application cannot instantiate this class.

2.5.12.1 ~CSIPServerTransaction

Function	<code>~CSIPServerTransaction();</code>
Description	Deletes the resources of CSIPServerTransaction.
Preconditions	None
Parameter	-
Return value	-
Postconditions	None
Exceptions	-

2.5.12.2 SendResponseL

Function	<code>void SendResponseL(CSIPResponseElements *aElements);</code>
Description	Sends a response and ends the transaction; the user cannot send a 100 Trying response.
Preconditions	<code>aElements != 0</code> <code>State() == CSIPTransactionBase::ETrying State() == CSIPTransactionBase::EProceeding</code> <code>CSIPConnection::State() == CSIPConnection::EActive</code> <code>ResponseAllowed() == ETrue</code>
Parameter	<code>CSIPResponseElements* aElements</code> : Contains Status Code, optional Reason Phrase, and optional SIP message headers and body. Ownership is transferred.
Return value	-
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.12.3 RequestElements

Function	<code>const CSIPRequestElements* RequestElements() const;</code>
Description	Gets the request elements.
Preconditions	None
Parameter	-
Return value	<code>const CSIPRequestElements*</code> : Request elements. Ownership is not transferred.
Postconditions	None
Exceptions	-

2.5.12.4 ResponseAllowed

Function	TBool ResponseAllowed() const;
Description	Checks if the response sending is allowed for this server transaction. For instance, response is not allowed in an ACK transaction.
Preconditions	None
Parameter	-
Return value	TBool: ETrue if response sending is allowed, otherwise EFalse.
Postconditions	None
Exceptions	-

2.5.13 CSIPSubscribeDialogAssoc

This class is for managing SIP SUBSCRIBE dialog associations. It provides services for creating, using, and terminating SIP SUBSCRIBE dialog associations. It is possible to have several CSIPSubscribeDialogAssoc instances that are associated with the same CSIPDialog.

2.5.13.1 NewL

Function	static CSIPSubscribeDialogAssoc* NewL(CSIPDialog& aDialog);
Description	Creates an instance of CSIPSubscribeDialogAssoc. Should be used if response to the SIP request to be sent will create a SIP dialog association.
Preconditions	None
Parameter	CSIPDialog& aDialog: A dialog to be associated with.
Return value	CSIPSubscribeDialogAssoc*: New instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.13.2 NewLC

Function	static CSIPSubscribeDialogAssoc* NewLC(CSIPDialog& aDialog);
Description	Creates an instance of CSIPSubscribeDialogAssoc and adds the pointer to the cleanup stack. Should be used if response to the SIP request to be sent will create a SIP dialog association.
Preconditions	None
Parameter	CSIPDialog& aDialog: A dialog to be associated with.
Return value	CSIPSubscribeDialogAssoc*: New instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.13.3 NewL

Function	<code>static CSIPSubscribeDialogAssoc* NewL(CSIPConnection& aConnection, CSIPFromHeader* aFrom, CSIPToHeader* aTo, CURIBase* aRemoteUri=0, CSIPContactHeader* aContact=0);</code>
Description	Creates an instance of CSIPSubscribeDialogAssoc.
Preconditions	<code>aFrom != 0</code> <code>aTo != 0</code> <code>aFrom->HasTag() == EFalse</code> <code>aTo->HasTag() == EFalse</code>
Parameter	<code>CSIPConnection& aConnection</code>: A SIP connection to be used with the dialog association. <code>CSIPFromHeader* aFrom</code>: Originator's address; the ownership is transferred. <code>CSIPToHeader* aTo</code>: Recipient's address; the ownership is transferred. <code>CURIBase* aRemoteURI</code>: A remote target URI. If not defined, the recipient's address will be used for remote-URI construction; the ownership is transferred. <code>CSIPContactHeader* aContact</code>: A contact to be used in dialog creation. Must be given only if user intends to redirect future requests; the ownership is transferred.
Return value	<code>CSIPSubscribeDialogAssoc*</code> : New instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.13.4 NewLC

Function	<code>static CSIPSubscribeDialogAssoc* NewLC(CSIPConnection& aConnection, CSIPFromHeader* aFrom, CSIPToHeader* aTo, CURIBase* aRemoteUri=0, CSIPContactHeader* aContact=0);</code>
Description	Creates an instance of CSIPSubscribeDialogAssoc and adds the pointer to the cleanup stack.
Preconditions	<code>aFrom != 0</code> <code>aTo != 0</code> <code>aFrom->HasTag() == EFalse</code> <code>aTo->HasTag() == EFalse</code>
Parameter	<code>CSIPConnection& aConnection</code>: A SIP connection to be used with the dialog association. <code>CSIPFromHeader* aFrom</code>: Originator's address; the ownership is transferred. <code>CSIPToHeader* aTo</code>: Recipient's address; the ownership is transferred. <code>CURIBase* aRemoteURI</code>: A remote target URI. If not defined, the recipient's address will be used for remote-URI construction; the ownership is transferred. <code>CSIPContactHeader* aContact</code>: A contact to be used in dialog creation. Must be given only if user intends to redirect future requests; the ownership is transferred.
Return value	<code>CSIPSubscribeDialogAssoc*</code> : New instance.

Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.13.5 NewL

Function	<code>static CSIPSubscribeDialogAssoc* NewL(CSIPConnection& aConnection, CSIPToHeader* aTo, MSIPRegistrationContext& aContext, CSIPFromHeader* aFrom = 0, CURIBase* aRemoteUri = 0, CSIPContactHeader* aContact = 0);</code>
Description	Creates an instance of CSIPSubscribeDialogAssoc.
Preconditions	<code>aTo != 0</code> <code>aFrom->HasTag() == EFalse</code> <code>aTo->HasTag() == EFalse</code> <code>aContext.IsActive() == Etrue</code>
Parameter	CSIPConnection& aConnection: A SIP connection to be used with dialog association. CSIPToHeader& aTo: Recipient's address; the ownership is transferred. MSIPRegistrationContext& aContext: Used for selecting outbound proxy and originator's address (AOR) and contact. CSIPFromHeader& aFrom: Originator's address. If not defined it will be constructed using registration context (user's AOR); the ownership is transferred. CURIBase* aRemoteURI: A remote target URI. If not defined, the recipient's address will be used for request-URI construction; the ownership is transferred. CSIPContactHeader* aContact: A contact to be used in dialog creation. Must be given only if user intends to redirect future requests; the ownership is transferred.
Return value	CSIPSubscribeDialogAssoc*: New instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.13.6 NewLC

Function	<code>static CSIPSubscribeDialogAssoc* NewLC(CSIPConnection& aConnection, CSIPToHeader* aTo, MSIPRegistrationContext& aContext, CSIPFromHeader* aFrom = 0, CURIBase* aRemoteUri = 0, CSIPContactHeader* aContact = 0);</code>
Description	Creates an instance of CSIPSubscribeDialogAssoc and adds the pointer to the cleanup stack.
Preconditions	<code>aTo != 0</code> <code>aFrom->HasTag() == EFalse</code> <code>aTo->HasTag() == EFalse</code> <code>aContext.IsActive() == Etrue</code>
Parameter	CSIPConnection& aConnection: A SIP connection to be used with dialog association. CSIPToHeader& aTo: Recipient's address; the ownership is transferred. MSIPRegistrationContext& aContext: Used for selecting outbound proxy and originator's address (AOR) and contact.

	CSIPFromHeader& aFrom: Originator's address. If not defined it will be constructed using registration context (user's AOR); the ownership is transferred. CURIBase* aRemoteURI: A remote target URI. If not defined, the recipient's address will be used for request-URI construction; the ownership is transferred. CSIPContactHeader* aContact: A contact to be used in dialog creation. Must be given only if user intends to redirect future requests; the ownership is transferred.
Return value	CSIPSubscribeDialogAssoc*: New instance.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.13.7 ~CSIPSubscribeDialogAssoc

Function	~CSIPSubscribeDialogAssoc();
Description	Deletes the resources of CSIPSubscribeDialogAssoc. This does not cause an (un)SUBSCRIBE request to be sent to the network.
Preconditions	None
Parameter	-
Return value	-
Postconditions	None
Exceptions	-

2.5.13.8 SendSubscribeL

Function	CSIPClientTransaction* SendSubscribeL(CSIPMessageElements* aElements=0, CSIPRefresh* aSIPRefresh=0);
Description	Creates SUBSCRIBE and sends it to the remote target. A 101-199 or 2xx response will create a dialog association in case of the first SUBSCRIBE request within this dialog association. If the first call to SendSubscribeL specified aSIPRefresh, the application can update the refreshed SUBSCRIBE by calling SendSubscribeL again. For the subsequent calls, the application must not give aSIPRefresh, since the SUBSCRIBE is already being refreshed.
Preconditions	Dialog().Connection().State() == CSIPConnection::EActive Dialog().State() == CSIPDialog::EInit Dialog().State() == CSIPDialog::EConfirmed
Parameter	CSIPMessageElements* aElements: Optional SIP message headers and body. Ownership is transferred. CSIPRefresh* aSIPRefresh: If set, the transaction will be refreshed at a given interval. The interval must be defined by including Expires-header. Ownership is transferred.
Return value	CSIPClientTransaction* SUBSCRIBE SIP transaction. Ownership is transferred.

Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.13.9 SendUnsubscribeL

Function	CSIPClientTransaction* SendUnsubscribeL(CSIPMessageElements* aElements=0);
Description	Creates (un)SUBSCRIBE and sends it to the remote target. The possible associated refresh will be terminated as well. Any SIP response (2xx - 6xx) to this request removes a subscription.
Preconditions	Dialog().Connection().State() == CSIPConnection::EActive Dialog().State() == CSIPDialog::EInit Dialog().State() == CSIPDialog::EConfirmed
Parameter	CSIPMessageElements* aElements: Optional SIP message headers and body. Ownership is transferred.
Return value	CSIPClientTransaction*: SUBSCRIBE SIP transaction. Ownership is transferred.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.13.10 SIPRefresh

Function	const CSIPRefresh* SIPRefresh() const;
Description	Gets associated refresh in case the user has requested the refresh of the SIP subscription. Note that a refreshed SUBSCRIBE dialog association cannot be terminated or updated using the returned object.
Preconditions	None
Parameter	-
Return value	const CSIPRefresh*: Associated refresh or a 0-pointer if the user has not requested a refresh. Ownership is not transferred.
Postconditions	None
Exceptions	-

2.5.14 CSIPTransactionBase

CSIPtransactionBase is a base class for managing SIP transactions. It provides services for querying transaction type and its state. This class is an abstract class and cannot be instantiated.

2.5.14.1 ~CSIPTransactionBase

Function	~CSIPTransactionBase();
Description	Deletes the resources of CSIPTransactionBase.
Preconditions	None

Parameter	-
Return value	-
Postconditions	None
Exceptions	-

2.5.14.2 Type

Function	<code>CSIPTransactionBase::TType Type() const;</code>
Description	Gets the type of the SIP transaction.
Preconditions	None
Parameter	-
Return value	<code>CSIPTransactionBase::TType</code> : SIP transaction type.
Postconditions	None
Exceptions	-

2.5.14.3 State

Function	<code>TState State() const;</code>
Description	Gets the state of the SIP transaction.
Preconditions	None
Parameter	-
Return value	<code>TState</code> : SIP transaction state.
Postconditions	None
Exceptions	-

2.5.14.4 IsSIPClientTransaction

Function	<code>TBool IsSIPClientTransaction() const;</code>
Description	Checks if the actual object is a client- or server-side transaction.
Preconditions	None
Parameter	-
Return value	<code>TBool</code> : <code>ETrue</code> if object is of type <code>CSIPClientTransaction</code> , otherwise <code>EFalse</code> .
Postconditions	None
Exceptions	-

2.5.14.5 operator==

Function	TBool operator==(const CSIPTransactionBase& aTransaction) const;
Description	Compares this object to another object also having CSIPTransactionBase base class.
Preconditions	None
Parameter	const CSIPTransactionBase& aTransaction: A CSIPTransactionBase object to compare.
Return value	TBool: ETrue if the objects are equal, otherwise EFalse.
Postconditions	None
Exceptions	-

2.5.15 MSIPConnectionObserver

This is an interface to be implemented by the users of CSIPConnection. The interface allows the application to receive requests, responses, connection state notifications, and error notifications from the SIP stack.

Note: The client must not delete the related CSIPConnection object during the execution of the interface function.

2.5.15.1 ~MSIPConnectionObserver

Function	virtual ~MSIPConnectionObserver();
Description	Deletes the resources of MSIPConnectionObserver.
Preconditions	None
Parameter	-
Return value	-
Postconditions	None
Exceptions	-

2.5.15.2 IncomingRequest

Function	virtual void IncomingRequest(CSIPServerTransaction* aTransaction) = 0;
Description	A SIP request outside a dialog has been received from the network.
Preconditions	aTransaction != 0
Parameter	CSIPServerTransaction* aTransaction: SIP server transaction. The ownership is transferred.
Return value	-
Postconditions	None
Exceptions	-

2.5.15.3 IncomingRequest

Function	<code>virtual void IncomingRequest(CSIPServerTransaction* aTransaction, CSIPDialog& aDialog) = 0;</code>
Description	A SIP request within a dialog has been received from the network. The application must resolve the actual dialog association to which this request belongs.
Preconditions	<code>aTransaction != 0</code>
Parameter	<code>CSIPServerTransaction* aTransaction</code> : SIP server transaction. The ownership is transferred. <code>CSIPDialog& aDialog</code> : The dialog that this transaction belongs to.
Return value	-
Postconditions	None
Exceptions	-

2.5.15.4 IncomingResponse

Function	<code>virtual void IncomingResponse(CSIPClientTransaction& aTransaction) = 0;</code>
Description	A SIP response has been received from the network.
Preconditions	None
Parameter	<code>CSIPClientTransaction& aTransaction</code> : The transaction that received the response. Contains response elements.
Return value	-
Postconditions	None
Exceptions	-

2.5.15.5 IncomingResponse

Function	<code>virtual void IncomingResponse(CSIPClientTransaction& aTransaction, CSIPDialogAssocBase& aDialogAssoc) = 0;</code>
Description	A SIP response that belongs to an existing dialog association or creates a dialog association. The SIP response has been received from the network
Preconditions	None
Parameter	<code>CSIPClientTransaction& aTransaction</code> : The transaction that received the response. Contains response elements. <code>CSIPDialogAssocBase& aDialogAssoc</code> : A dialog association.
Return value	-
Postconditions	None
Exceptions	-

2.5.15.6 IncomingResponse

Function	<code>virtual void IncomingResponse(CSIPClientTransaction& aTransaction, CSIPInviteDialogAssoc* aDialogAssoc) = 0;</code>
Description	Multiple SIP responses have been received to a single INVITE due to the forking proxy. Note that each response creates a separate INVITE dialog association. Multiple responses can arrive until the SIP stack completes the UAC core INVITE transaction. If the client deletes the INVITE transaction after the first SIP response, the other possible responses will be consumed by the implementation.
Preconditions	None
Parameter	CSIPClientTransaction& aTransaction: The transaction that received the response. Contains response elements. CSIPInviteDialogAssoc& aDialogAssoc: An INVITE dialog association; the ownership is transferred.
Return value	-
Postconditions	None
Exceptions	-

2.5.15.7 IncomingResponse

Function	<code>virtual void IncomingResponse(CSIPClientTransaction& aTransaction, CSIPRegistration& aRegistration) = 0;</code>
Description	A SIP response related to a registration binding or an error response that is related to a registration binding has been received from the network.
Preconditions	None
Parameter	CSIPClientTransaction& aTransaction: The transaction that received the response. Contains response elements. CSIPRegistration& aRegistration: The registration this transaction belongs to.
Return value	-
Postconditions	None
Exceptions	-

2.5.15.8 ErrorOccured

Function	<code>virtual void ErrorOccured (TInt aError, CSIPTransactionBase& aTransaction) = 0;</code>
Description	An asynchronous error has occurred in the stack related to the request indicated by the given transaction.
Preconditions	None
Parameter	TInt aError: A system-wide or SIP error code. CSIPTransactionBase& aTransaction: The failed transaction.
Return value	-

Postconditions	None
Exceptions	-

2.5.15.9 *ErrorOccured*

Function	<code>virtual void ErrorOccured(TInt aError, CSIPClientTransaction& aTransaction, CSIPRegistration& aRegistration) = 0;</code>
Description	An asynchronous error has occurred in the stack related to the request indicated by the given transaction.
Preconditions	None
Parameter	TInt aError: A system-wide or SIP error code. CSIPClientTransaction& aTransaction: The failed transaction. CSIPRegistration& aRegistration: The failed registration.
Return value	-
Postconditions	None
Exceptions	-

2.5.15.10 *ErrorOccured*

Function	<code>virtual void ErrorOccured(TInt aError, CSIPTransactionBase& aTransaction, CSIPDialogAssocBase& aDialogAssoc) = 0;</code>
Description	An asynchronous error has occurred related to a request within an existing dialog.
Preconditions	None
Parameter	TInt aError: A system-wide or SIP error code. CSIPTransactionBase& aTransaction: The failed transaction. CSIPDialogAssocBase& aDialogAssoc: The failed dialog association.
Return value	-
Postconditions	None
Exceptions	-

2.5.15.11 *ErrorOccured*

Function	<code>virtual void ErrorOccured(TInt aError, CSIPRefresh& aSIPRefresh) = 0;</code>
Description	An asynchronous error has occurred related to a refresh.
Preconditions	None
Parameter	TInt aError: A system-wide or SIP error code. CSIPRefresh& aSIPRefresh: The original refresh object.
Return value	-

Postconditions	None
Exceptions	-

2.5.15.12 *ErrorOccured*

Function	<code>virtual void ErrorOccured(TInt aError, CSIPRegistration& aRegistration) = 0;</code>
Description	An asynchronous error has occurred related to a periodical refresh that relates to a registration.
Preconditions	None
Parameter	TInt aError: A system-wide or SIP error code. CSIPRegistration& aRegistration: Associated registration.
Return value	-
Postconditions	None
Exceptions	-

2.5.15.13 *ErrorOccured*

Function	<code>virtual void ErrorOccured(TInt aError, CSIPDialogAssocBase& aDialogAssoc) = 0;</code>
Description	An asynchronous error has occurred related to a periodical refresh that belongs to a SIP dialog association.
Preconditions	None
Parameter	TInt aError: A system-wide or SIP error code. CSIPDialogAssocBase& aDialogAssoc: SIP dialog association.
Return value	-
Postconditions	None
Exceptions	-

2.5.15.14 *InviteCompleted*

Function	<code>virtual void InviteCompleted(CSIPClientTransaction& aTransaction) = 0;</code>
Description	SIP stack has completed a UAC core INVITE transaction 64*T1 seconds after the reception of the first 2xx response. No more 2xx responses can be received to the issued single INVITE. If the INVITE transaction does not create a dialog, or the INVITE transaction encounters an error, this event will not be sent.
Preconditions	None
Parameter	CSIPClientTransaction& aTransaction: The completed UAC core INVITE transaction.
Return value	-

Postconditions	None
Exceptions	-

2.5.15.15 *ConnectionStateChanged*

Function	<code>virtual void ConnectionStateChanged(CSIPConnection::TState aState) = 0;</code>
Description	Connection state has changed. If the connection state has changed to CSIPConnection::EInactive or CSIPConnection::EUnavailable, the SIP stack has removed all stand-alone SIP refreshes, registrations, and dialog associations that the client requested to refresh. Client may reissue refresh requests (stand-alone, registration, or dialog association related) when the connection becomes CSIPConnection::EActive again. The SIP stack also terminates all pending SIP client transactions and no errors are reported back to the client about the terminated transactions or about removed refreshes in order to avoid event flood.
Preconditions	None
Parameter	CSIPConnection::TState aState: Indicates the current connection state.
Return value	-
Postconditions	None
Exceptions	-

2.5.16 MSIPObserver

This is an interface to be implemented by the user of the CSIP class. This interface allows the application to receive a SIP request from a SIP connection that has not been initialized by the user.

2.5.16.1 *~MSIPObserver*

Function	<code>virtual ~MSIPObserver();</code>
Description	Deletes the resources of MSIPObserver.
Preconditions	None
Parameter	-
Return value	-
Postconditions	None
Exceptions	-

2.5.16.2 IncomingRequest

Function	<code>virtual void IncomingRequest(TUint32 aIapId, CSIPServerTransaction* aTransaction) = 0;</code>
Description	A SIP request has been received from the network, and the application has not yet created a CSIPConnection for the IAP in question.
Preconditions	<code>aTransaction != 0</code>
Parameter	TUint32 aIapId: The IAP identifier from which the SIP request was received. CSIPServerTransaction* aTransaction: Transaction containing local address, remote address of the SIP request, as well as optional SIP message method, headers, and body. The ownership is transferred.
Return value	-
Postconditions	None
Exceptions	-

2.5.16.3 TimedOut

Function	<code>virtual void TimedOut(CSIPServerTransaction& aTransaction) = 0;</code>
Description	The received SIP request timed-out and it has been destroyed. This will be called if the user fails to create a SIP connection and does not send an appropriate SIP response.
Preconditions	None
Parameter	CSIPServerTransaction& aTransaction: The timed-out transaction.
Return value	-
Postconditions	None
Exceptions	-

2.5.17 MSIPRegistrationContext

This is the interface for providing information for the SIP request and dialog creation.

2.5.17.1 ~MSIPRegistrationContext

Function	<code>virtual ~MSIPRegistrationContext();</code>
Description	Deletes the resources of MSIPRegistrationContext.
Preconditions	None
Parameter	-
Return value	-
Postconditions	None
Exceptions	-

2.5.17.2 *IsContextActive*

Function	<code>virtual TBool IsContextActive() const = 0;</code>
Description	Checks if the registration context can be used for creating SIP messages/dialogs.
Preconditions	None
Parameter	-
Return value	TBool: ETrue if the context can be used, otherwise EFalse.
Postconditions	None
Exceptions	-

2.5.17.3 *ContextId*

Function	<code>virtual TUint32 ContextId() const = 0;</code>
Description	Gets the context ID. This function is for internal use only.
Preconditions	None
Parameter	-
Return value	TUint32: Context ID.
Postconditions	None
Exceptions	-

2.5.18 SIPHttpDigest

This is the class for managing SIP HTTP Digest security settings. SIPHttpDigest provides functions for setting and removing HTTP Digest security-mechanism-related parameters.

2.5.18.1 *SetParametersL*

Function	<code>static void SetParametersL(CSIP& aSIP, const TDesC8& aServer, const TDesC8& aRealm, const TDesC8& aUsername, const TDesC8& aPasswd);</code>
Description	Sets parameters for the particular server and realm.
Preconditions	None
Parameter	CSIP& aSIP: A handle to a SIP server. const TDesC8& aServer: Server's FQDN or IP address. const TDesC8& aRealm: Server's realm. const TDesC8& aUsername: User's name. const TDesC8& aPasswd: User's password for the given server's realm.
Return value	-
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.18.2 RemoveParametersL

Function	static void RemoveParametersL(CSIP& aSIP, const TDesC8& aServer, const TDesC8& aRealm);
Description	Removes parameters for the particular server and realm.
Preconditions	None
Parameter	CSIP& aSIP: A handle to a SIP server. const TDesC8& aServer: Server's FQDN or IP address. const TDesC8& aRealm: Server's realm.
Return value	-
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.6 Code Architecture

The services offered by the SIP Client API are provided as a dynamically linked library called `sipclient.dll`. The header files of the API are located in `/epoc32/include` and are as follows:

<code>sip.h</code>	class CSIP
<code>sipclienttransaction.h</code>	class CSIPClientTransaction
<code>sipconnection.h</code>	class CSIPConnectio
<code>sipconnectionobserver.h</code>	class MSIPConnectionObserver
<code>sipdialog.h</code>	class CSIPDialog
<code>sipdialogassocbase.h</code>	class CSIPDialogAssocBase
<code>siphttpdigest.h</code>	class SIPHttpDigest
<code>sipinvitedialogassoc.h</code>	class CSIPInviteDialogAssoc
<code>sipmessageelements.h</code>	class CSIPMessageElements
<code>sipobserver.h</code>	class MSIPObserver
<code>siprefresh.h</code>	class CSIPRefresh
<code>siprefreshassociation.h</code>	For internal use
<code>sipregistration.h</code>	class CSIPRegistration
<code>sipregistrationcontext.h</code>	class MSIPRegistrationContext
<code>siprequestelements.h</code>	class CSIPRequestElements
<code>sipresponseelements.h</code>	class CSIPResponseElements
<code>sipsecurity.h</code>	For internal use

sipservertransaction.h	class CSIPServerTransaction
sipsubscribedialogassoc.h	class CSIPSubscribeDialogAssoc
siptransactionbase.h	class CSIPTransactionBase
transactionassociation.h	For internal use

3 Index

3.1 List of Constants and Enums

3.1.1 class CSIP

```

/** SIP specific errors */
enum TError
{
    /** SIP message was malformed */
    EMalformedMessage = -17700,
    /** Registrar responded with a different AOR (in the To-header) */
    ERegistrarRespondedWithDifferentAOR = -17701,
    /** Registrar modified Contact-header */
    ERegistrarRespondedWithDifferentSipAddressInContact = -17702,
    /** Registrar responded without contact header */
    ERegistrarRespondedWithoutContact = -17703,
    /** Request pending */
    ERequestPending = -17704,
    /** The other end responded with expires value zero */
    EResponseWithExpiresZero = -17705,
    /** The other end responded with a different tag in To-header */
    EResponseWithDifferentRemoteTag = -17706,
    /** Transport port is already in use */
    EPortAlreadyInUse = -17707,
    /** The action can't be performed in the current transaction state */
    EInvalidTransactionState = -17708,
    /** Not allowed in dialogs current state */
    EInvalidDialogState = -17709,
    /** Dialogs creation request invalid */
    EInvalidDialogCreationRequest = -17710,
    /** Dialogs creation response invalid */
    EInvalidDialogCreationResponse = -17711,
    /** Response in dialogs invalid */
    EInvalidResponseInDialogs = -17712,
    /** Request in dialogs invalid */
    EInvalidRequestInDialogs = -17713,
    /** Overlapping request pending in a dialog */
    EOverlappingDialogRequestPending = -17714,
    /** Sending a SIP message failed. For example ICMP error occurred */
    ETransportFailure = -17715,
    /** No ACK was received after sending a 2xx response */
    ENoAckReceived = -17716,
    /** Not allowed in registration's current state */
    EInvalidRegistrationState = -17717,
    /** The contact given did not contain user part */
    EInvalidContact = -17718,
    /** Object can't access a resource, since that has been deleted by
    user. User is expected to delete this object as it can no longer be
    used. */
    EResourceNotAvailable = -17719,
    /** DNS query for the remote address failed */
    EResolvingFailure = -17720,
    /** Authentication with a server failed */
    EForbidden = -17721,
    /** Maximum number of allowed SigComp compartments exceeded */
    EMaxCompartmentsInUse = -17722,
    /** Refresh initiated client transaction was terminated with a 3xx,
    4xx, 5xx or 6xx response.*/
    ETerminatedWithResponse = -17723
};

```

3.1.2 class CSIPConnection

```

/** SIP minimum value for the expiration in seconds*/
const TInt KSIPMinExpirationValue=60;

/** Connection states */
enum TState
{
    /** Connection initializing */
    EInit=1,
    /** Connection active */
    EActive,
    /** Connection suspended */
    ESuspended,
    /** Connection is inactive */
    EInactive,
    /** Connection is permanently unavailable */
    EUnavailable
};

```

3.1.3 class CSIPDialog

```

/** Dialog states */
enum TState
{
    /** Initilized state */
    EInit,
    /** Early state */
    EEarly,
    /** Confirmed state */
    EConfirmed,
    /** Terminated state */
    ETerminated
};

```

3.1.4 class CSIPInviteDialogAssoc

```

/** INVITE dialog association type*/
const TInt KSIPInviteDialogAssoc=2;

```

3.1.5 class CSIPRefresh

```

/** SIP refresh states */
enum TState
{
    /** SIP refresh is inactive */
    EInactive,
    /** SIP refresh active */
    EActive,
    /** SIP refresh is terminated */
    ETerminated
};

```

3.1.6 class CSIPSubscribeDialogAssoc

```

/** SUBSCRIBE dialog association type*/
const TInt KSIPSubscribeDialogAssoc=4;

```

3.1.7 class CSIPTransactionBase

```

/** SIP transaction type */
enum TType
{

```

```

    /** INVITE transaction */
    EINVITE,
    /** SUBSCRIBE transaction */
    ESUBSCRIBE,
    /** NOTIFY transaction */
    ENOTIFY,
    /** MESSAGE transaction */
    EMESSAGE,
    /** CANCEL transaction */
    ECANCEL,
    /** BYE transaction */
    EBYE,
    /** OPTIONS transaction */
    EOPTIONS,
    /** PRACK transaction */
    EPRACK,
    /** UPDATE transaction */
    EUPDATE,
    /** ACK transaction */
    EACK,
    /** Register transaction */
    EREGISTER,
    /** Extension transaction */
    EEXTENSION
};

/** SIP transaction state */
enum TState
{
    /** Trying state */
    ETrying,
    /** Calling state */
    ECalling,
    /** Proceeding state */
    EProceeding,
    /** Completed state */
    ECompleted,
    /** Confirmed state */
    EConfirmed,
    /** Terminated state */
    ETerminated
};

```

3.2 Alphabetical List of Classes and Functions

3.2.1 class CSIP

```

IMPORT_C static CSIP* NewL(const TUid& aUid, MSIPObserver&
    aObserver);
IMPORT_C static CSIP* NewLC(const TUid& aUid, MSIPObserver&
    aObserver);
IMPORT_C ~CSIP();
IMPORT_C CDesC8Array* SupportedSecurityMechanismsL() const;
IMPORT_C TBool IsSigCompSupportedL() const;
IMPORT_C CSIPConnection* Connection(TUint32 aIapId) const;

```

3.2.2 class CSIPClientTransaction

```

IMPORT_C ~CSIPClientTransaction();
IMPORT_C const CSIPResponseElements* ResponseElements() const;
IMPORT_C CSIPClientTransaction* Cancell();
IMPORT_C const CSIPRefresh* Refresh() const;
IMPORT_C virtual TBool CancelAllowed() const;

```

3.2.3 class CSIPConnection

```

IMPORT_C static CSIPConnection* NewL(CSIP& aSIP, TUint32 aIapId,
    MSIPConnectionObserver& aObserver);
IMPORT_C static CSIPConnection* NewLC(CSIP& aSIP, TUint32 aIapId,
    MSIPConnectionObserver& aObserver);
IMPORT_C ~CSIPConnection();
IMPORT_C TState State() const;
IMPORT_C CSIPClientTransaction* SendRequestL(CSIPRequestElements*
    aElements, const MSIPRegistrationContext& aContext);
IMPORT_C CSIPClientTransaction* SendRequestL(CSIPRequestElements*
    aElements, const MSIPRegistrationContext& aContext,
    CSIPRefresh& aRefresh);
IMPORT_C CSIPClientTransaction* SendRequestL(CSIPRequestElements*
    aElements);
IMPORT_C CSIPClientTransaction* SendRequestL(CSIPRequestElements*
    aElements, CSIPRefresh& aRefresh);
IMPORT_C CSIPClientTransaction*
    FetchRegistrationsL(CSIPRequestElements* aElements);
IMPORT_C TUint32 IapId() const;

```

3.2.4 class CSIPDialog

```

IMPORT_C CSIPDialog::TState State() const;
IMPORT_C const RPointerArray<CSIPDialogAssocBase>&
    SIPDialogAssociations() const;
IMPORT_C const MSIPRegistrationContext*
    RegistrationContext() const;
IMPORT_C TBool IsAssociated(const CSIPDialogAssocBase&
    aDialogAssoc) const;
IMPORT_C CSIPConnection* Connection();
IMPORT_C const CSIPConnection* Connection() const;
IMPORT_C const CSIPFromHeader& FromHeader() const;
IMPORT_C const CSIPToHeader& ToHeader() const;
IMPORT_C const CURIBase* RemoteURI() const;
IMPORT_C TBool operator==(const CSIPDialog& aDialog) const;

```

3.2.5 class CSIPInviteDialogAssoc

```

IMPORT_C static CSIPInviteDialogAssoc* NewL(
    CSIPDialog& aSIPDialog);
IMPORT_C static CSIPInviteDialogAssoc* NewLC(
    CSIPDialog& aSIPDialog);
IMPORT_C static CSIPInviteDialogAssoc* NewL(CSIPDialog& aDialog,
    CSIPServerTransaction& aTransaction);
IMPORT_C static CSIPInviteDialogAssoc* NewLC(CSIPDialog& aDialog,
    CSIPServerTransaction& aTransaction);
IMPORT_C static CSIPInviteDialogAssoc* NewL(
    CSIPServerTransaction& aTransaction);
IMPORT_C static CSIPInviteDialogAssoc* NewLC(
    CSIPServerTransaction& aTransaction);
IMPORT_C static CSIPInviteDialogAssoc* NewL(CSIPConnection&
    aConnection,
    CSIPFromHeader* aFrom, CSIPToHeader* aTo,
    CURIBase* aRemoteUri=0, CSIPContactHeader* aContact=0);
IMPORT_C static CSIPInviteDialogAssoc* NewLC(CSIPConnection&
    aConnection,
    CSIPFromHeader* aFrom, CSIPToHeader* aTo,
    CURIBase* aRemoteUri=0, CSIPContactHeader* aContact=0);
IMPORT_C static CSIPInviteDialogAssoc* NewL(CSIPConnection&
    aConnection,
    CSIPToHeader* aTo, const MSIPRegistrationContext& aContext,
    CSIPFromHeader* aFrom = 0, CURIBase* aRemoteUri = 0,
    CSIPContactHeader* aContact = 0);
IMPORT_C static CSIPInviteDialogAssoc* NewLC(CSIPConnection&
    aConnection, CSIPToHeader* aTo, const SIPRegistrationContext&
    aContext, CSIPFromHeader* aFrom = 0,

```

```

    CURIBase* aRemoteUri = 0, CSIPContactHeader* aContact = 0);
IMPORT_C ~CSIPInviteDialogAssoc();
IMPORT_C CSIPClientTransaction* SendInviteL(CSIPMessageElements*
    aElements=0);
IMPORT_C CSIPClientTransaction* SendPrackL(CSIPMessageElements*
    aElements=0);
IMPORT_C CSIPClientTransaction* SendUpdateL(CSIPMessageElements*
    aElements=0);
IMPORT_C void SendAckL(const CSIPClientTransaction& aTransaction,
    CSIPMessageElements* aElements=0);
IMPORT_C CSIPClientTransaction* SendByeL(CSIPMessageElements*
    aElements=0);

```

3.2.6 class CSIPMessageElements

```

IMPORT_C static CSIPMessageElements* NewL();
IMPORT_C static CSIPMessageElements* NewLC();
IMPORT_C virtual ~CSIPMessageElements();
IMPORT_C void SetUserHeadersL(RPointerArray<CSIPHeaderBase>&
    aHeaders);
IMPORT_C const RPointerArray<CSIPHeaderBase>&
    UserHeaders() const;
IMPORT_C void SetContentL(HBufC8* aContent,
    CSIPContentTypeHeader* aContentType);
IMPORT_C const TDesC8& Content() const;
IMPORT_C const CSIPContentTypeHeader* ContentType() const;
IMPORT_C HBufC8* ExtractContent();

```

3.2.7 class CSIPRefresh

```

IMPORT_C static CSIPRefresh* NewL();
IMPORT_C static CSIPRefresh* NewLC();
IMPORT_C ~CSIPRefresh();
IMPORT_C CSIPRefresh::TState State() const;
IMPORT_C const CSIPClientTransaction* SIPTransaction() const;
IMPORT_C CSIPClientTransaction* TerminateL(CSIPMessageElements*
    aElements=0);
IMPORT_C CSIPClientTransaction* UpdateL(CSIPMessageElements*
    aElements=0);
IMPORT_C TBool operator==(const CSIPRefresh& aRefresh) const;

```

3.2.8 class CSIPRegistration

```

IMPORT_C static CSIPRegistration* NewL(CSIPConnection&
    aConnection,
    CSIPToHeader* aAor, CSIPContactHeader* aContact,
    CSIPRefresh* aRefresh=0, CSIPRouteHeader* aOutboundProxy=0,
    CSIPURI* aRemoteUri=0, CSIPFromHeader* aFrom=0);
IMPORT_C static CSIPRegistration* NewLC(CSIPConnection&
    aConnection,
    CSIPToHeader* aAor, CSIPContactHeader* aContact,
    CSIPRefresh* aRefresh=0, CSIPRouteHeader* aOutboundProxy=0,
    CSIPURI* aRemoteUri=0, CSIPFromHeader* aFrom=0);
IMPORT_C ~CSIPRegistration();
IMPORT_C CSIPClientTransaction* RegisterL(CSIPMessageElements*
    aElements=0);
IMPORT_C CSIPClientTransaction* DeregisterL(CSIPMessageElements*
    aElements=0);
IMPORT_C CSIPClientTransaction* UpdateL(CSIPMessageElements*
    aElements);
IMPORT_C CSIPClientTransaction* UpdateL(TUint aExpirationValue,
    CSIPMessageElements* aElements=0);
IMPORT_C CSIPConnection* Connection();
IMPORT_C const CSIPConnection* Connection() const;
IMPORT_C void SetOutboundProxyL(CSIPRouteHeader* aOutboundProxy);
IMPORT_C const CSIPToHeader& Aor() const;

```

```

IMPORT_C const CSIPContactHeader& ContactHeader() const;
IMPORT_C const CSIPRouteHeader* OutboundProxy() const;
IMPORT_C const CSIPRefresh* SIPRefresh() const;
IMPORT_C TBool operator==(const CSIPRegistration&
    aRegistration) const;

```

3.2.9 class CSIPRequestElements

```

IMPORT_C static CSIPRequestElements* NewL(CSIPToHeader*
    aToHeader);
IMPORT_C static CSIPRequestElements* NewLC(CSIPToHeader*
    aToHeader);
IMPORT_C ~CSIPRequestElements();
IMPORT_C void SetToHeaderL(CSIPToHeader* aTo);
IMPORT_C const CSIPToHeader* ToHeader() const;
IMPORT_C void SetFromHeaderL(CSIPFromHeader* aFrom);
IMPORT_C const CSIPFromHeader* FromHeader() const;
IMPORT_C void SetRemoteURIL(CURIBase* aURI);
IMPORT_C const CURIBase* RemoteURI() const;
IMPORT_C void SetMethodL(const TDesC8& aMethod);
IMPORT_C const TDesC8& Method() const;
IMPORT_C const CSIPMessageElements& MessageElements() const;
IMPORT_C CSIPMessageElements& MessageElements();

```

3.2.10 class CSIPResponseElements

```

IMPORT_C static CSIPResponseElements* NewL(TUint aStatusCode,
    const TDesC8& aReasonPhrase);
IMPORT_C static CSIPResponseElements* NewLC(TUint aStatusCode,
    const TDesC8& aReasonPhrase);
IMPORT_C ~CSIPResponseElements();
IMPORT_C void SetStatusCodeL(TUint aStatusCode);
IMPORT_C TUint StatusCode() const;
IMPORT_C void SetReasonPhraseL(const TDesC8& aReasonPhrase);
IMPORT_C const TDesC8& ReasonPhrase() const;
IMPORT_C const CSIPFromHeader* FromHeader() const;
IMPORT_C const CSIPToHeader* ToHeader() const;
IMPORT_C const CSIPCSeqHeader* CSeqHeader() const;
IMPORT_C const CSIPMessageElements& MessageElements() const;
IMPORT_C CSIPMessageElements& MessageElements();

```

3.2.11 class CSIPServerTransaction

```

IMPORT_C ~CSIPServerTransaction();
IMPORT_C void SendResponseL(CSIPResponseElements *aElements);
IMPORT_C const CSIPRequestElements* RequestElements() const;
IMPORT_C TBool ResponseAllowed() const;

```

3.2.12 class CSIPSubscribeDialogAssoc

```

IMPORT_C static CSIPSubscribeDialogAssoc* NewL(CSIPDialog&
    aDialog);
IMPORT_C static CSIPSubscribeDialogAssoc* NewLC(CSIPDialog&
    aDialog);
IMPORT_C static CSIPSubscribeDialogAssoc* NewL(CSIPConnection&
    aConnection, CSIPFromHeader* aFrom, CSIPToHeader* aTo,
    CURIBase* aRemoteUri=0, CSIPContactHeader* aContact=0);
IMPORT_C static CSIPSubscribeDialogAssoc* NewLC(CSIPConnection&
    aConnection, CSIPFromHeader* aFrom, CSIPToHeader* aTo,
    CURIBase* aRemoteUri=0, CSIPContactHeader* aContact=0);
IMPORT_C static CSIPSubscribeDialogAssoc* NewL(CSIPConnection&
    aConnection, CSIPToHeader* aTo, MSIPRegistrationContext&
    aContext, CSIPFromHeader* aFrom = 0, CURIBase*
    aRemoteUri = 0, CSIPContactHeader* aContact = 0);
IMPORT_C static CSIPSubscribeDialogAssoc* NewLC(CSIPConnection&

```

```

        aConnection, CSIPToHeader* aTo, MSIPRegistrationContext&
        aContext, CSIPFromHeader* aFrom = 0, CURIBase*
        aRemoteUri = 0, CSIPContactHeader* aContact = 0);
IMPORT_C ~CSIPSubscribeDialogAssoc();
IMPORT_C CSIPClientTransaction* SendSubscribeL(
    CSIPMessageElements* aElements=0,
    CSIPRefresh* aSIPRefresh=0);
IMPORT_C CSIPClientTransaction* SendUnsubscribeL(
    CSIPMessageElements* aElements=0);
IMPORT_C const CSIPRefresh* SIPRefresh() const;

```

3.2.13 class CSIPTransactionBase

```

virtual ~CSIPTransactionBase();
IMPORT_C CSIPTransactionBase::TType Type() const;
IMPORT_C TState State() const;
IMPORT_C TBool IsSIPClientTransaction() const;
IMPORT_C TBool operator==(const CSIPTransactionBase&
    aTransaction) const;

```

3.2.14 class MSIPConnectionObserver

```

virtual void IncomingRequest(CSIPServerTransaction*
    aTransaction) = 0;
virtual void IncomingRequest(CSIPServerTransaction*
    aTransaction, CSIPDialog& aDialog) = 0;
virtual void IncomingResponse(CSIPClientTransaction&
    aTransaction) = 0;
virtual void IncomingResponse(CSIPClientTransaction&
    aTransaction, CSIPDialogAssocBase& aDialogAssoc) = 0;
virtual void IncomingResponse(CSIPClientTransaction&
    aTransaction, CSIPInviteDialogAssoc* aDialogAssoc) = 0;
virtual void IncomingResponse(CSIPClientTransaction&
    aTransaction, CSIPRegistration& aRegistration) = 0;
virtual void ErrorOccured(TInt aError, CSIPTransactionBase&
    aTransaction) = 0;
virtual void ErrorOccured(TInt aError, CSIPClientTransaction&
    aTransaction, CSIPRegistration& aRegistration) = 0;
virtual void ErrorOccured(TInt aError, CSIPTransactionBase&
    aTransaction, CSIPDialogAssocBase& aDialogAssoc) = 0;
virtual void ErrorOccured(TInt aError,
    CSIPRefresh& aSIPRefresh) = 0;
virtual void ErrorOccured(TInt aError, CSIPRegistration&
    aRegistration) = 0;
virtual void ErrorOccured(TInt aError, CSIPDialogAssocBase&
    aDialogAssoc) = 0;
virtual void InviteCompleted(CSIPClientTransaction&
    aTransaction) = 0;
virtual void ConnectionStateChanged(CSIPConnection::TState
    aState) = 0;

```

3.2.15 class MSIPObserver

```

virtual void IncomingRequest(TUint32 aIapId,
    CSIPServerTransaction* aTransaction) = 0;
virtual void TimedOut(CSIPServerTransaction& aTransaction) = 0;

```

3.2.16 class MSIPRegistrationContext

```

virtual TBool IsContextActive() const = 0;
virtual TUint32 ContextId() const = 0;

```

3.2.17 class SIPHttpDigest

```
IMPORT_C static void SetParametersL(CSIP& aSIP,  
    const TDesC8& aServer,  
    const TDesC8& aRealm, const TDesC8& aUsername,  
    const TDesC8& aPasswd);  
IMPORT_C static void RemoveParametersL(CSIP& aSIP,  
    const TDesC8& aServer, const TDesC8& aRealm);
```

4 Terms and Abbreviations

Term or abbreviation	Meaning
3GPP	3 rd Generation Partnership Project
AOR	Address Of Record
API	Application Programming Interface
FQDN	Full Qualified Domain Name
IAP	Internet Access Point
IMS	IP Multimedia System
SIP	Session Initiation Protocol
SIP stack server side	The shared server in the client/server architecture of the SIP stack implementation. Not to be confused with the remote server of a SIP transaction.

5 References

[1] SIP Codec API Specification, <http://www.forum.nokia.com/sip>.